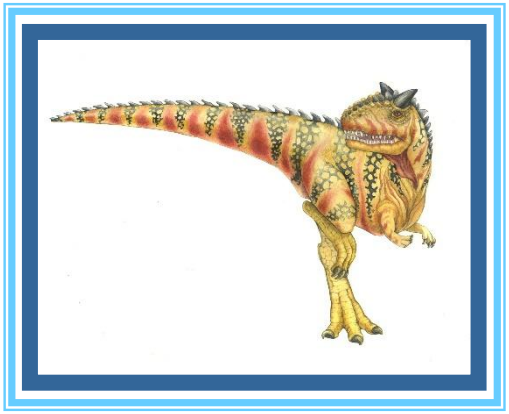
CS2303&2310: Operating Systems

School of Computer Science
Chentao Wu 吴晨涛
wuct@cs.sjtu.edu.cn

上海交通大學
SHANGHAI JIAO TONG UNIVERSITY

Chapter 8: Deadlocks



Chapter 8: Deadlocks

System Model

Deadlock in Multithreaded Applications

Deadlock Characterization

Methods for Handling Deadlocks

Deadlock Prevention

Deadlock Avoidance

Deadlock Detection

Recovery from Deadlock

Chapter Objectives

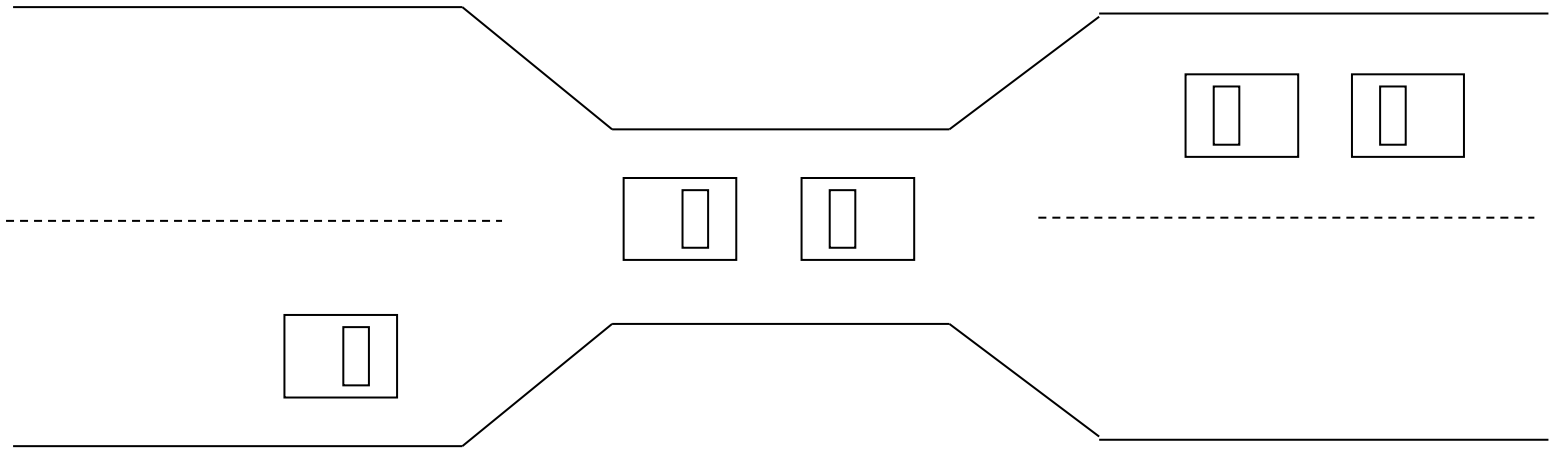
- Illustrate how deadlock can occur when mutex locks are used
- Define the four necessary conditions that characterize deadlock
- Identify a deadlock situation in a resource allocation graph
- Evaluate the four different approaches for preventing deadlocks
- Apply the banker's algorithm for deadlock avoidance
- Apply the deadlock detection algorithm
- Evaluate approaches for recovering from deadlock

The Deadlock Problem

- A set of blocked processes each holding a resource and waiting to acquire a resource held by another process in the set
- Example
  - System has 2 disk drives
  - P_1 and P_2 each hold one disk drive and each needs another one
- Example
  - semaphores *A* and *B*, initialized to 1
  -

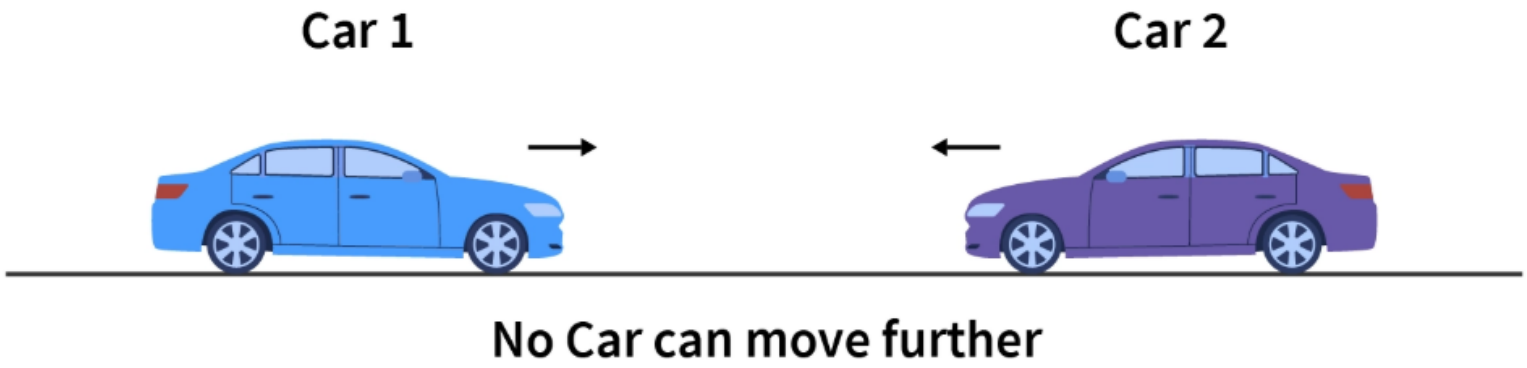
P_0	P_1
wait (A);	wait(B)
wait (B);	wait(A)

Bridge Crossing Example



- Traffic only in one direction
- Each section of a bridge can be viewed as a resource
- If a deadlock occurs, it can be resolved if one car backs up (preempt resources and rollback)
- Several cars may have to be backed up if a deadlock occurs
- Starvation is possible
- Note – Most OSes do not prevent or deal with deadlocks

Bridge Crossing Example



System Model

System consists of resources

Resource types $R_1, R_2, \ldots, R_m$

CPU cycles, memory space, I/O devices

Each resource type R_i has W_i instances.

Each process utilizes a resource as follows:

request

use

release

Deadlock in Multithreaded Application

Two mutex locks are created an initialized:

```
pthread_mutex_t first_mutex;
pthread_mutex_t second_mutex;

pthread_mutex_init(&first_mutex,NULL);
pthread_mutex_init(&second_mutex,NULL);
```

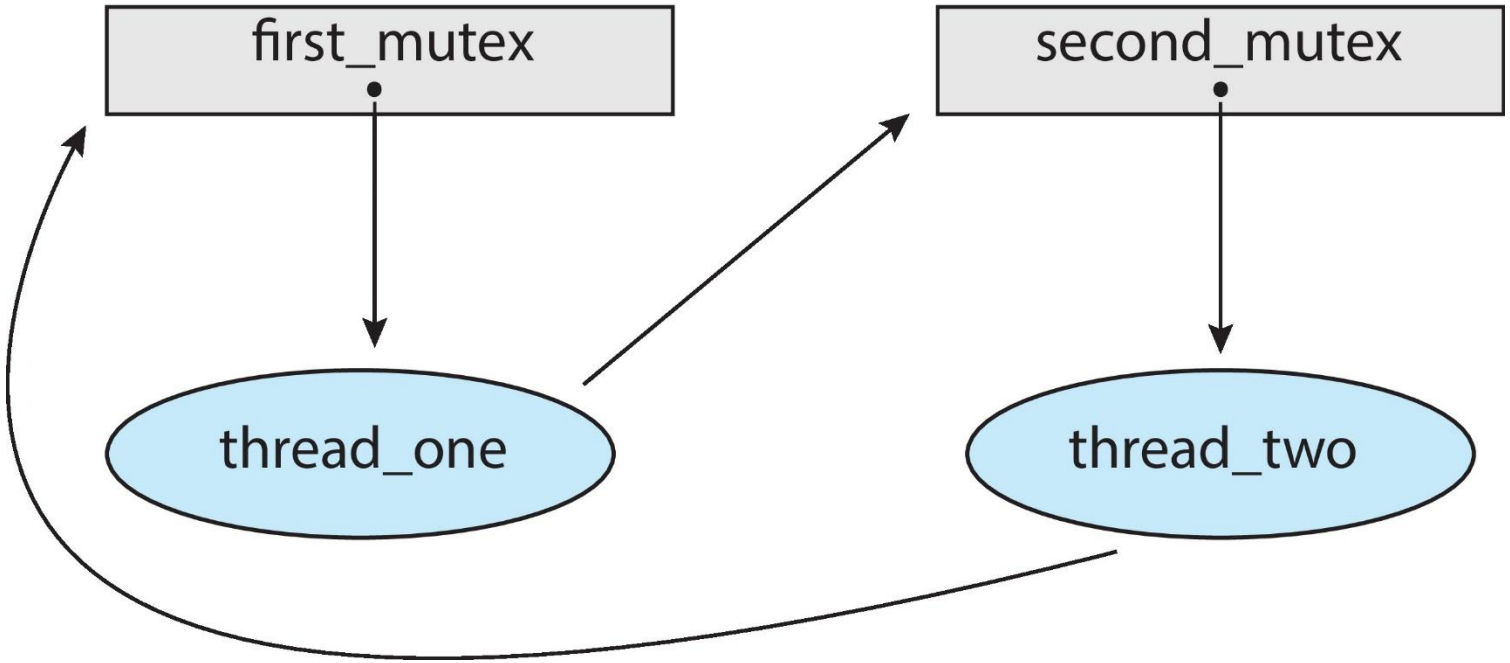
Deadlock in Multithreaded Application

```
/* thread_one runs in this function */
void *do_work_one(void *param)
{
   pthread_mutex_lock(&first_mutex);
   pthread_mutex_lock(&second_mutex);
   /**
    * Do some work
    */
   pthread_mutex_unlock(&second_mutex);
   pthread_mutex_unlock(&first_mutex);

   pthread_exit(0);
}

/* thread_two runs in this function */
void *do_work_two(void *param)
{
   pthread_mutex_lock(&second_mutex);
   pthread_mutex_lock(&first_mutex);
   /**
    * Do some work
    */
   pthread_mutex_unlock(&first_mutex);
   pthread_mutex_unlock(&second_mutex);

   pthread_exit(0);
}
```

Deadlock in Multithreaded Application

Deadlock is possible if thread 1 acquires **`first_mutex`** and thread 2 acquires **`second_mutex`**. Thread 1 then waits for **`second_mutex`** and thread 2 waits for **`first_mutex`**.

Can be illustrated with a **resource allocation graph**:



Deadlock Characterization

Deadlock can arise if four conditions hold simultaneously.

- **Mutual exclusion**: only one process at a time can use a resource
- **Hold and wait**: a process holding at least one resource is waiting to acquire additional resources held by other processes
- **No preemption**: a resource can be released only voluntarily by the process holding it, after that process has completed its task
- **Circular wait**: there exists a set $\{P_0, P_1, \ldots, P_n\}$ of waiting processes such that P_0 is waiting for a resource that is held by P_1, P_1 is waiting for a resource that is held by P_2, …, P_{n-1} is waiting for a resource that is held by P_n, and P_n is waiting for a resource that is held by P_0.

Resource-Allocation Graph

A set of vertices *V* and a set of edges *E*.

V is partitioned into two types:

$P = \{P_1, P_2, \ldots, P_n\}$, the set consisting of all the processes in the system

$R = \{R_1, R_2, \ldots, R_m\}$, the set consisting of all resource types in the system

request edge – directed edge $P_i \rightarrow R_j$

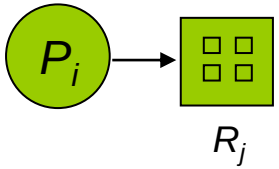
assignment edge – directed edge $R_j \rightarrow P_i$

Resource-Allocation Graph (Cont.)

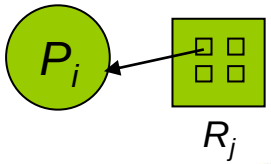
- Process

- Resource Type with 4 instances

- P_i requests instance of R_j



- P_i is holding an instance of R_j



Resource Allocation Graph Example

One instance of R1

Two instances of R2

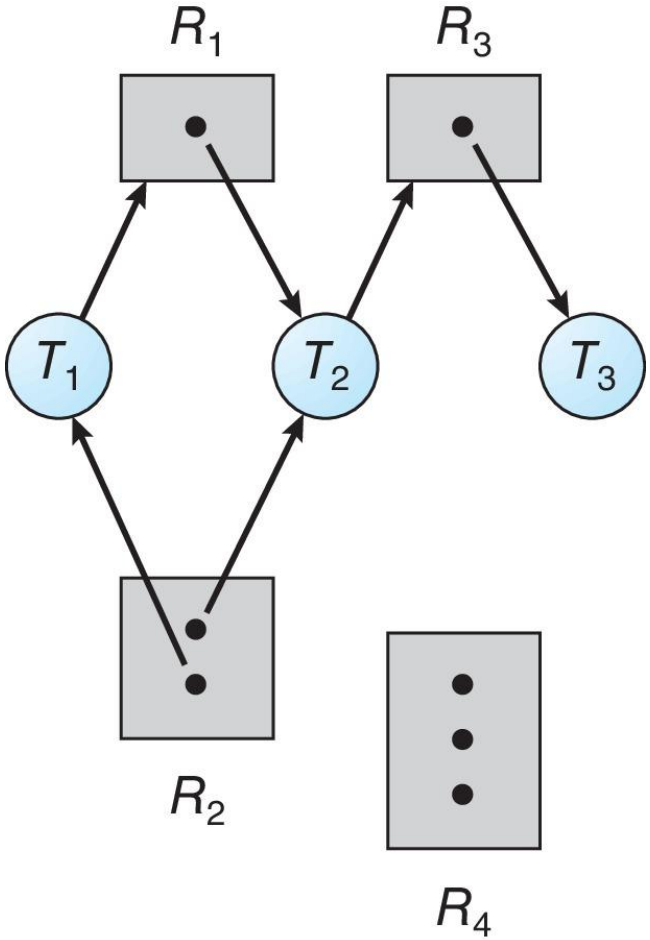
One instance of R3

Three instance of R4

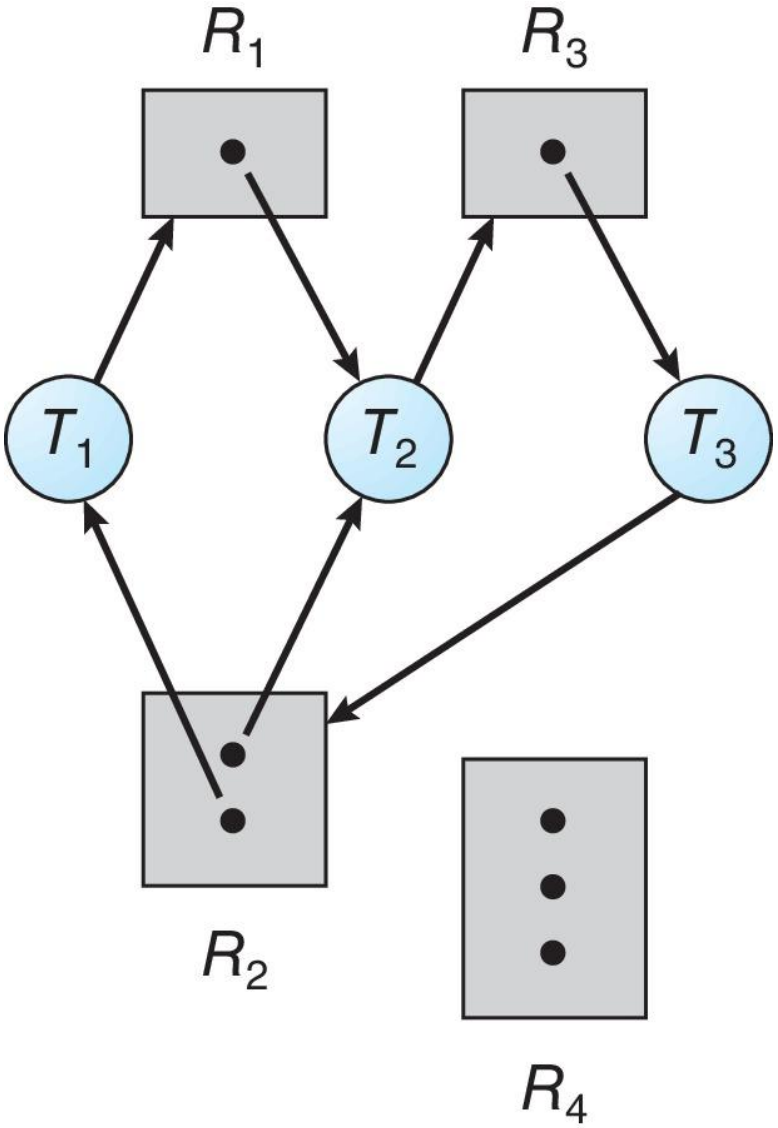
T1 holds one instance of R2 and is waiting for an instance of R1

T2 holds one instance of R1, one instance of R2, and is waiting for an instance of R3

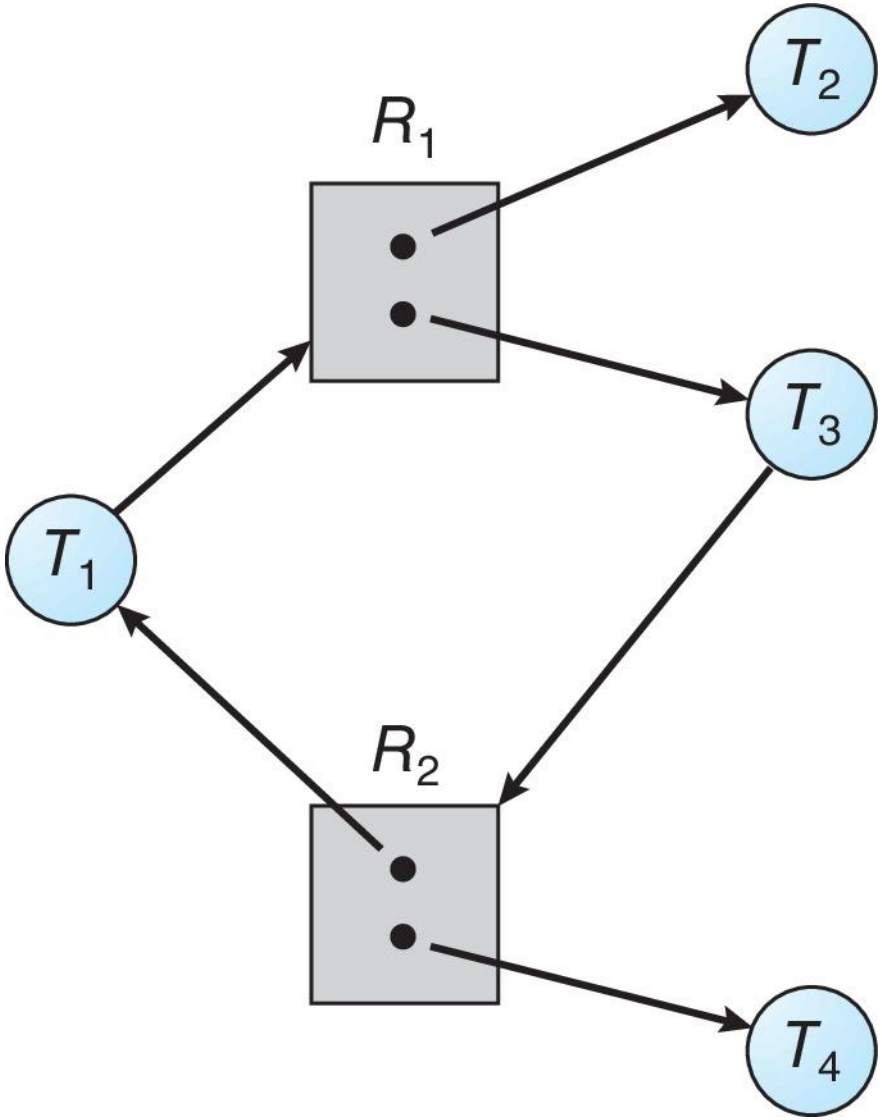
T3 is holds one instance of R3



Resource Allocation Graph With A Deadlock



Graph With A Cycle But No Deadlock



Basic Facts

If graph contains no cycles $\Rightarrow$ no deadlock

If graph contains a cycle $\Rightarrow$

- if only one instance per resource type, then deadlock
- if several instances per resource type, possibility of deadlock

Methods for Handling Deadlocks

Ensure that the system will ***never*** enter a deadlock state:

- Deadlock prevention
- Deadlock avoidance

Allow the system to enter a deadlock state and then recover

Ignore the problem and pretend that deadlocks never occur in the system.

Deadlock Prevention

Invalidate one of the four necessary conditions for deadlock:

Mutual Exclusion – not required for sharable resources (e.g., read-only files); must hold for non-sharable resources

Hold and Wait – must guarantee that whenever a process requests a resource, it does not hold any other resources

Require process to request and be allocated all its resources before it begins execution, or allow process to request resources only when the process has none allocated to it.

Low resource utilization; starvation possible

Deadlock Prevention (Cont.)

No Preemption –

- If a process that is holding some resources requests another resource that cannot be immediately allocated to it, then all resources currently being held are released
- Preempted resources are added to the list of resources for which the process is waiting
- Process will be restarted only when it can regain its old resources, as well as the new ones that it is requesting

Circular Wait – impose a total ordering of all resource types, and require that each process requests resources in an increasing order of enumeration

Circular Wait

Invalidating the circular wait condition is most common.

Simply assign each resource (i.e. mutex locks) a unique number.

Resources must be acquired in order

If:

`first_mutex = 1`
`second_mutex = 5`

code for **`thread_two`** could not be written as follows:

```
/* thread_one runs in this function */
void *do_work_one(void *param)
{
   pthread_mutex_lock(&first_mutex);
   pthread_mutex_lock(&second_mutex);
   /**
    * Do some work
    */
   pthread_mutex_unlock(&second_mutex);
   pthread_mutex_unlock(&first_mutex);

   pthread_exit(0);
}

/* thread_two runs in this function */
void *do_work_two(void *param)
{
   pthread_mutex_lock(&second_mutex);
   pthread_mutex_lock(&first_mutex);
   /**
    * Do some work
    */
   pthread_mutex_unlock(&first_mutex);
   pthread_mutex_unlock(&second_mutex);

   pthread_exit(0);
}
```

Deadlock Avoidance

Requires that the system has some additional ***a priori*** information available

- Simplest and most useful model requires that each process declare the ***maximum number*** of resources of each type that it may need
- The deadlock-avoidance algorithm dynamically examines the resource-allocation state to ensure that there can never be a circular-wait condition
- Resource-allocation *state* is defined by the number of available and allocated resources, and the maximum demands of the processes

Deadlock Prevention (Cont.)

Condition	Approach	Is practically possible ?
Mutual Exclusion	Spooling	✗
Hold and wait	Request for all the resources initially	✗
No preemption	Snatch all the resources	✗
Circular wait	Assign priority to each resources and order resources numerically	✓

Safe State

When a process requests an available resource, system must decide if immediate allocation leaves the system in a safe state

System is in **safe state** if there exists a sequence <P_1, P_2, …, P_n> of ALL the processes in the systems such that for each P_i, the resources that P_i can still request can be satisfied by currently available resources + resources held by all the P_j, with $j < I$

That is:

- If P_i resource needs are not immediately available, then P_i can wait until all P_j have finished
- When P_j is finished, P_i can obtain needed resources, execute, return allocated resources, and terminate
- When P_i terminates, P_{i+1} can obtain its needed resources, and so on

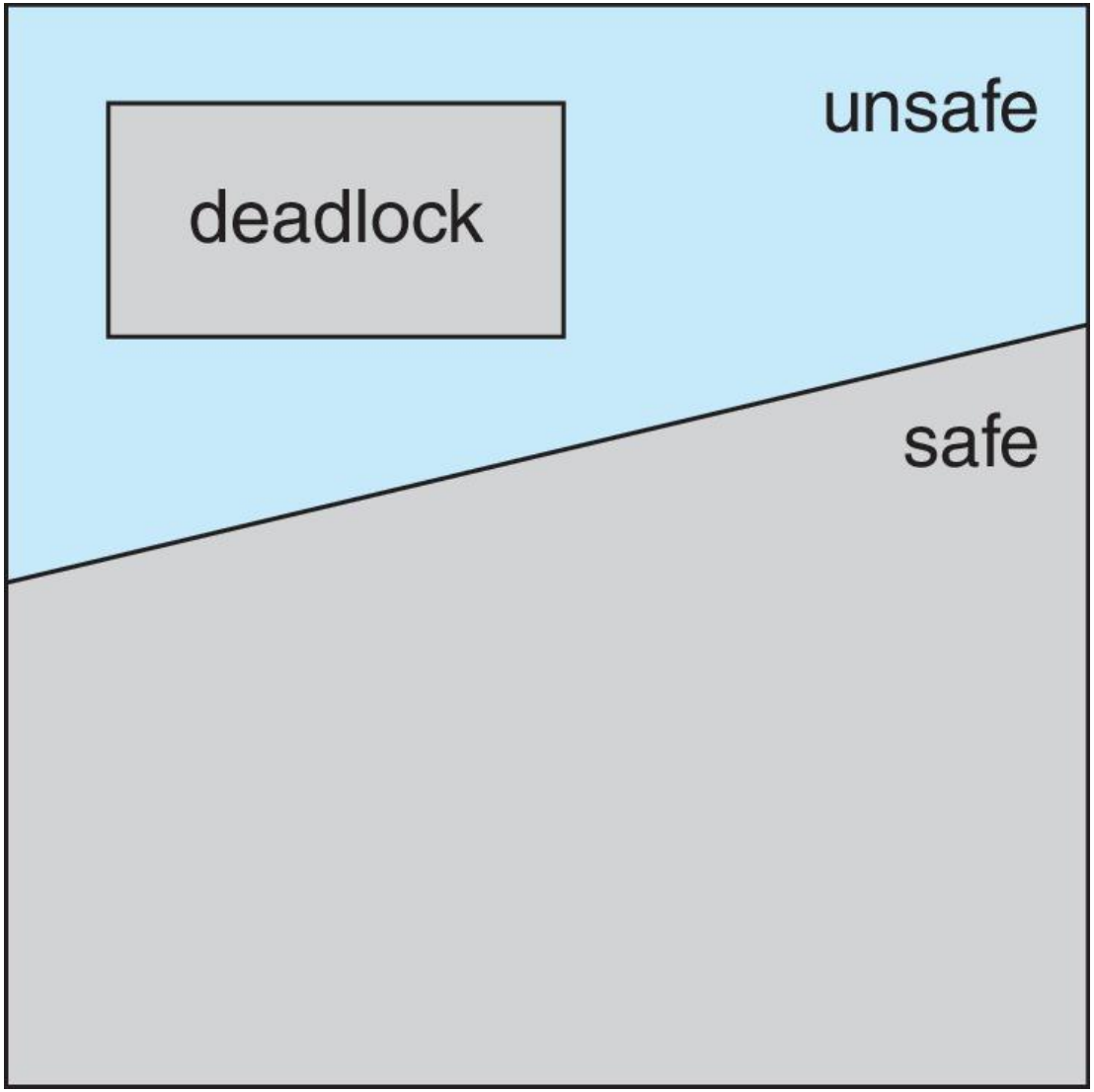
Basic Facts

If a system is in safe state $\Rightarrow$ no deadlocks

If a system is in unsafe state $\Rightarrow$ possibility of deadlock

Avoidance $\Rightarrow$ ensure that a system will never enter an unsafe state.

Safe, Unsafe, Deadlock State



Avoidance Algorithms

Single instance of a resource type

Use a resource-allocation graph

Multiple instances of a resource type

Use the Banker's Algorithm

Resource-Allocation Graph Scheme

Claim edge $P_i \rightarrow R_j$ indicated that process P_j may request resource R_j; represented by a dashed line

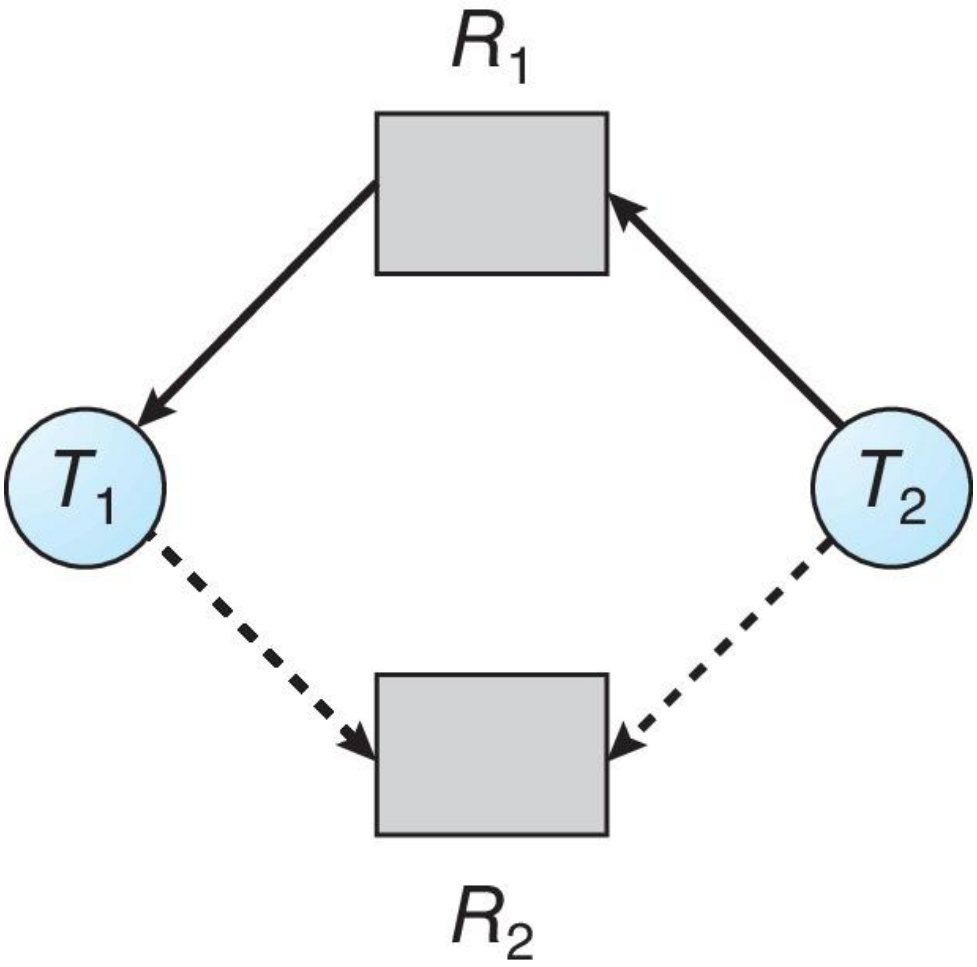
Claim edge converts to request edge when a process requests a resource

Request edge converted to an assignment edge when the resource is allocated to the process

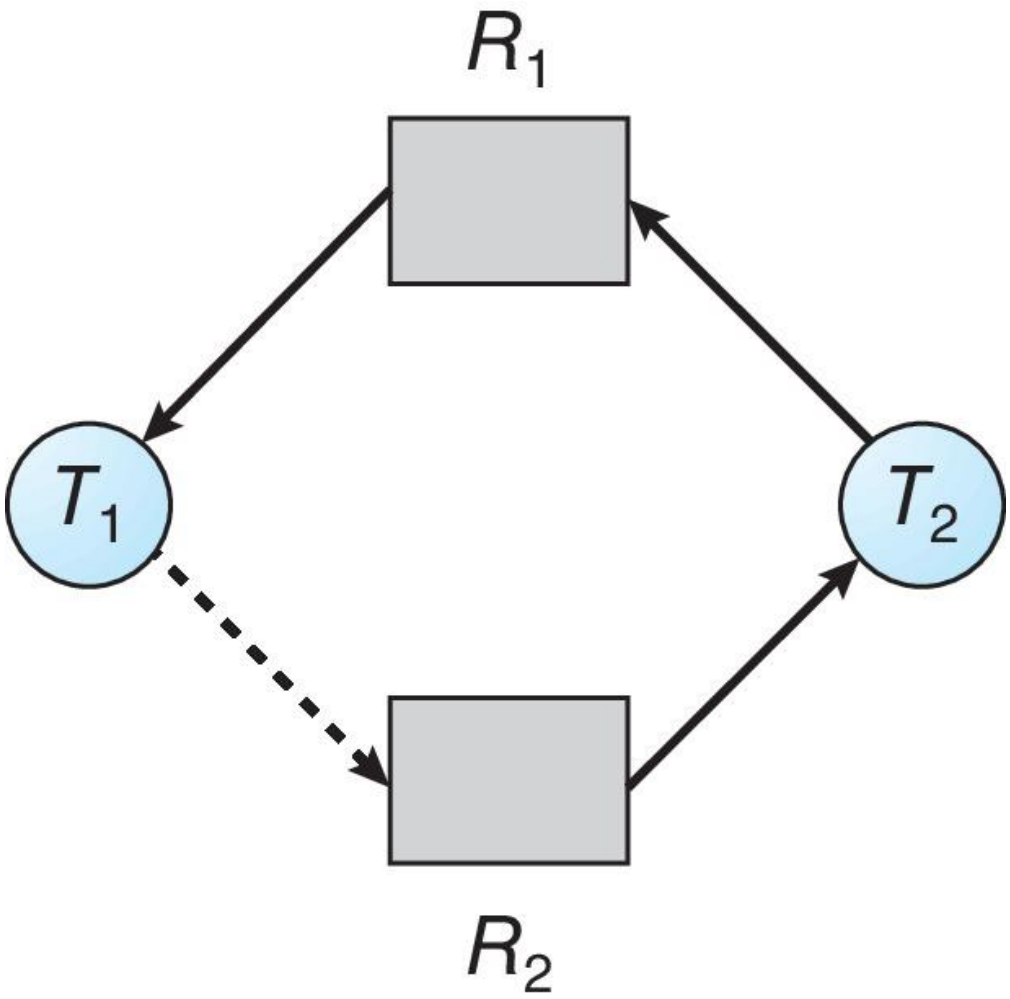
When a resource is released by a process, assignment edge reconverts to a claim edge

Resources must be claimed *a priori* in the system

Resource-Allocation Graph



Unsafe State In Resource-Allocation Graph



Resource-Allocation Graph Algorithm

Suppose that process P_i requests a resource R_j

The request can be granted only if converting the request edge to an assignment edge does not result in the formation of a cycle in the resource allocation graph

Banker's Algorithm

Multiple instances of resources

Each process must a priori claim maximum use

When a process requests a resource it may have to wait

When a process gets all its resources it must return them in a finite amount of time

Data Structures for the Banker's Algorithm

Let n = number of processes, and m = number of resources types.

Available: Vector of length m. If available $[j] = k$, there are k instances of resource type R_j available

Max: $n \times m$ matrix. If $Max\ [i,j] = k$, then process P_i may request at most k instances of resource type R_j

Allocation: $n \times m$ matrix. If Allocation$[i,j] = k$ then P_i is currently allocated k instances of R_j

Need: $n \times m$ matrix. If $Need[i,j] = k$, then P_i may need k more instances of R_j to complete its task

$$Need\ [i,j] = Max[i,j] – Allocation\ [i,j]$$

Safety Algorithm

1. Let ***Work*** and ***Finish*** be vectors of length *m* and *n*, respectively. Initialize:

   Work = Available

   ***Finish* [*i*] = *false* for *i* = 0, 1, …, *n*- 1**

2. Find an ***i*** such that both:

   (a) ***Finish* [*i*] = *false***

   (b) $\boldsymbol{Need_i \leq Work}$

   If no such ***i*** exists, go to step 4

3. $\boldsymbol{Work = Work + Allocation_i}$
   Finish*[*i*] = *true
   go to step 2

4. If ***Finish* [*i*] == *true*** for all ***i***, then the system is in a safe state

Resource-Request Algorithm for Process P_i

$\boldsymbol{Request_i}$ = request vector for process $\boldsymbol{P_i}$. If $\boldsymbol{Request_i}$ **[*j*] = *k*** then process $\boldsymbol{P_i}$ wants ***k*** instances of resource type $\boldsymbol{R_j}$

1. If $\boldsymbol{Request_i \leq Need_i}$ go to step 2. Otherwise, raise error condition, since process has exceeded its maximum claim
2. If $\boldsymbol{Request_i \leq Available}$, go to step 3. Otherwise $\boldsymbol{P_i}$ must wait, since resources are not available
3. Pretend to allocate requested resources to $\boldsymbol{P_i}$ by modifying the state as follows:

$$\boldsymbol{Available = Available - Request_i;}$$

$$\boldsymbol{Allocation_i = Allocation_i + Request_i;}$$

$$\boldsymbol{Need_i = Need_i - Request_i;}$$

If safe $\Rightarrow$ the resources are allocated to $\boldsymbol{P_i}$

If unsafe $\Rightarrow$ $\boldsymbol{P_i}$ must wait, and the old resource-allocation state is restored

Example of Banker's Algorithm

5 processes P_0 through P_4;

3 resource types:

A (10 instances), *B* (5instances), and *C* (7 instances)

Snapshot at time T_0:

	Allocation	*Max*	*Available*
	A B C	*A B C*	*A B C*
P_0	0 1 0	7 5 3	3 3 2
P_1	2 0 0	3 2 2	
P_2	3 0 2	9 0 2	
P_3	2 1 1	2 2 2	
P_4	0 0 2	4 3 3	

Example (Cont.)

The content of the matrix ***Need*** is defined to be ***Max – Allocation***

	Need
	A B C
P_0	7 4 3
P_1	1 2 2
P_2	6 0 0
P_3	0 1 1
P_4	4 3 1

The system is in a safe state since the sequence < P_1, P_3, P_4, P_2, P_0> satisfies safety criteria

Example: P_1 Request (1,0,2)

Check that Request $\leq$ Available (that is, $(1,0,2) \leq (3,3,2) \Rightarrow$ true

	Allocation	*Need*	*Available*
	A B C	*A B C*	*A B C*
P_0	0 1 0	7 4 3	2 3 0
P_1	3 0 2	0 2 0	
P_2	3 0 2	6 0 0	
P_3	2 1 1	0 1 1	
P_4	0 0 2	4 3 1	

Executing safety algorithm shows that sequence < $\boldsymbol{P_1}$, $\boldsymbol{P_3}$, $\boldsymbol{P_4}$, $\boldsymbol{P_0}$, $\boldsymbol{P_2}$> satisfies safety requirement

Can request for (3,3,0) by $\boldsymbol{P_4}$ be granted?

Can request for (0,2,0) by $\boldsymbol{P_0}$ be granted?

Deadlock Detection

Allow system to enter deadlock state

Detection algorithm

Recovery scheme

Single Instance of Each Resource Type

Maintain **wait-for** graph

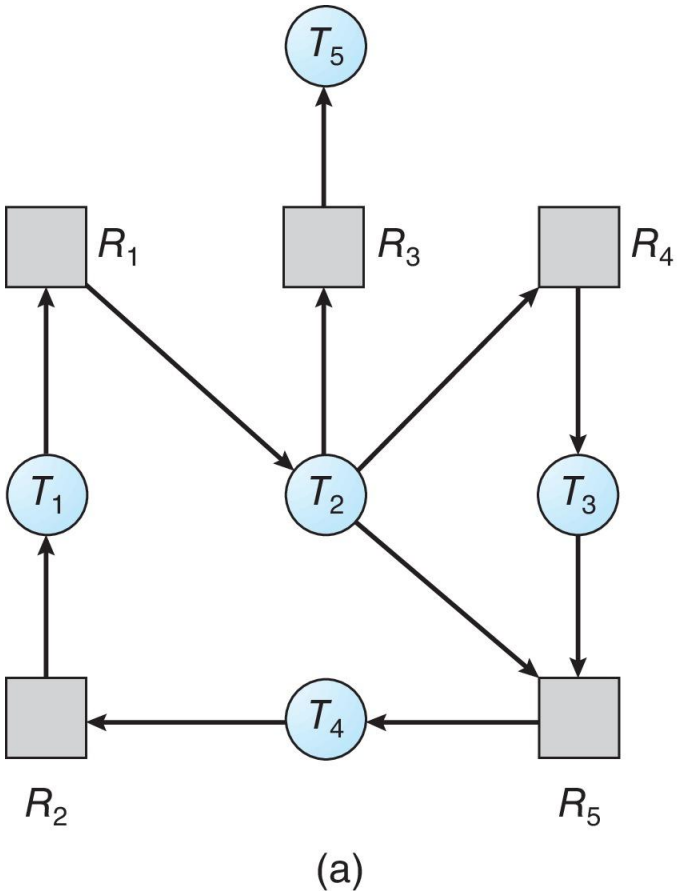
Nodes are processes

$P_i \rightarrow P_j$ if P_i is waiting for P_j

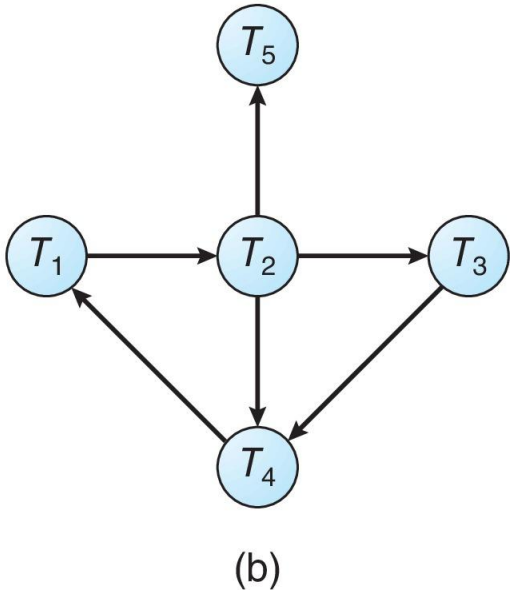
Periodically invoke an algorithm that searches for a cycle in the graph. If there is a cycle, there exists a deadlock

An algorithm to detect a cycle in a graph requires an order of n^2 operations, where n is the number of vertices in the graph

Resource-Allocation Graph and Wait-for Graph



Resource-Allocation Graph



Corresponding wait-for graph

Several Instances of a Resource Type

Available*:* A vector of length ***m*** indicates the number of available resources of each type

Allocation*:* An ***n* x *m*** matrix defines the number of resources of each type currently allocated to each process

Request*:* An ***n* x *m*** matrix indicates the current request of each process. If ***Request* [*i*][*j*] = *k***, then process P_i is requesting ***k*** more instances of resource type R_j.

Detection Algorithm

1. Let ***Work*** and ***Finish*** be vectors of length ***m*** and ***n***, respectively Initialize:

   (a) ***Work* = *Available***

   (b) For ***i* = 1,2, …, *n***, if $\boldsymbol{Allocation_i \neq 0}$, then ***Finish*[i] = *false***; otherwise, ***Finish*[i] = *true***

2. Find an index ***i*** such that both:

   (a) ***Finish*[*i*] == *false***

   (b) $\boldsymbol{Request_i \leq Work}$

   If no such ***i*** exists, go to step 4

Detection Algorithm (Cont.)

3. ***Work* = *Work* + *Allocation*$_i$**
   Finish*[*i*] = *true
   go to step 2

4. If ***Finish[i]* == *false***, for some ***i***, $1 \leq i \leq n$, then the system is in deadlock state. Moreover, if ***Finish*[*i*] == *false***, then P_i is deadlocked

Algorithm requires an order of O(m x n^2) operations to detect whether the system is in deadlocked state

Example of Detection Algorithm

Five processes P_0 through P_4; three resource types A (7 instances), *B* (2 instances), and *C* (6 instances)

Snapshot at time T_0:

	Allocation	*Request*	*Available*
	A B C	*A B C*	*A B C*
P_0	0 1 0	0 0 0	0 0 0
P_1	2 0 0	2 0 2	
P_2	3 0 3	0 0 0	
P_3	2 1 1	1 0 0	
P_4	0 0 2	0 0 2	

Sequence <P_0, P_2, P_3, P_1, P_4> will result in ***Finish[i] = true*** for all ***i***

Example (Cont.)

P_2 requests an additional instance of type C

	Request
	A B C
P_0	0 0 0
P_1	2 0 2
P_2	0 0 1
P_3	1 0 0
P_4	0 0 2

State of system?

- Can reclaim resources held by process P_0, but insufficient resources to fulfill other processes; requests
- Deadlock exists, consisting of processes P_1, P_2, P_3, and P_4

Detection-Algorithm Usage

When, and how often, to invoke depends on:

- How often a deadlock is likely to occur?
- How many processes will need to be rolled back?
  - one for each disjoint cycle

If detection algorithm is invoked arbitrarily, there may be many cycles in the resource graph and so we would not be able to tell which of the many deadlocked processes "caused" the deadlock.

Recovery from Deadlock: Process Termination

Abort all deadlocked processes

Abort one process at a time until the deadlock cycle is eliminated

In which order should we choose to abort?

1. Priority of the process
2. How long process has computed, and how much longer to completion
3. Resources the process has used
4. Resources process needs to complete
5. How many processes will need to be terminated
6. Is process interactive or batch?

Recovery from Deadlock: Resource Preemption

Selecting a victim – minimize cost

Rollback – return to some safe state, restart process for that state

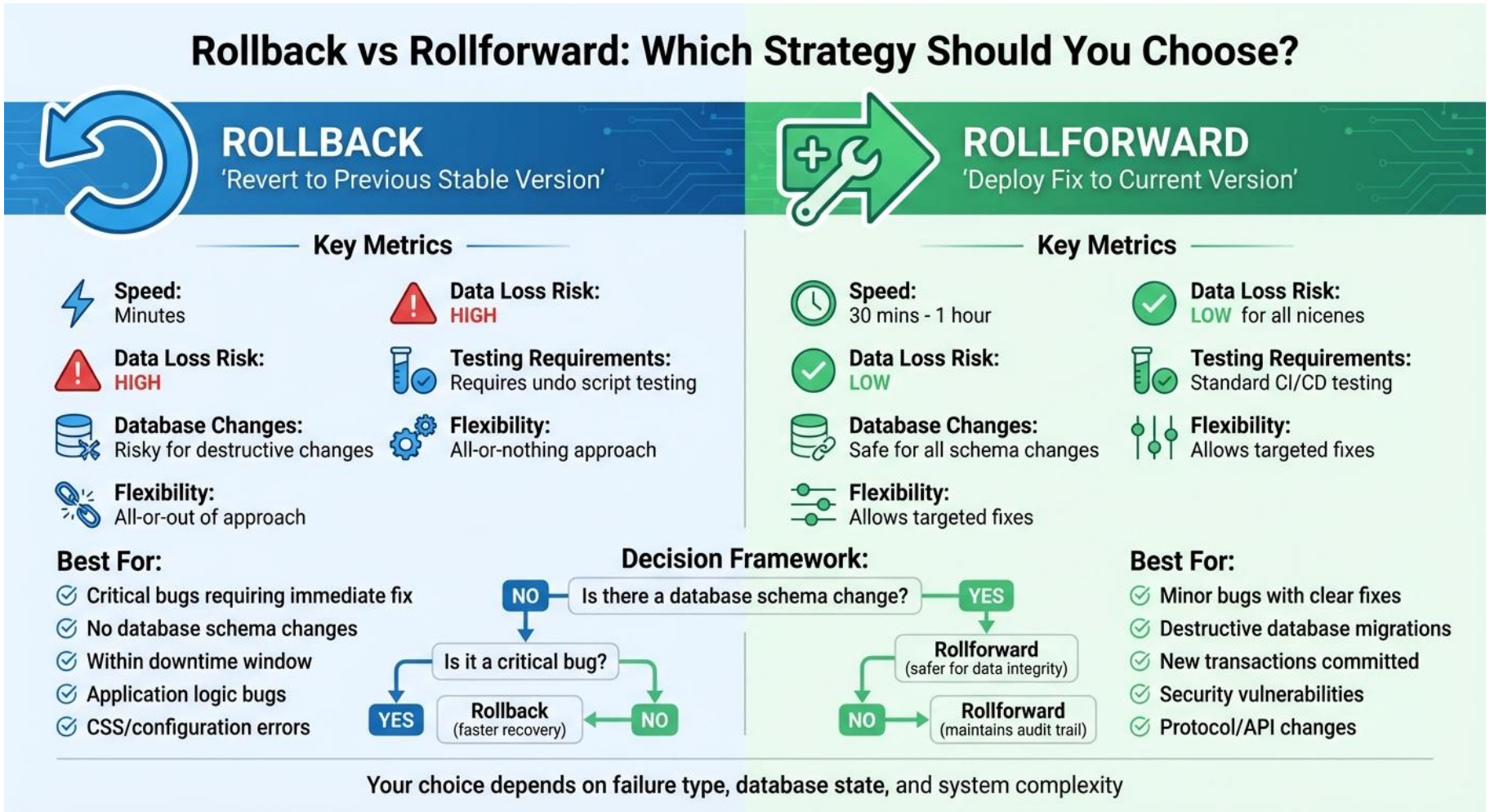
Starvation – same process may always be picked as victim, include number of rollback in cost factor

Rollback and Rollforward

Rollback means reverting to a previous stable version, which is often faster but can risk data loss.

Rollforward fixes the issue by deploying a new version, preserving data and deployment history but taking more time.

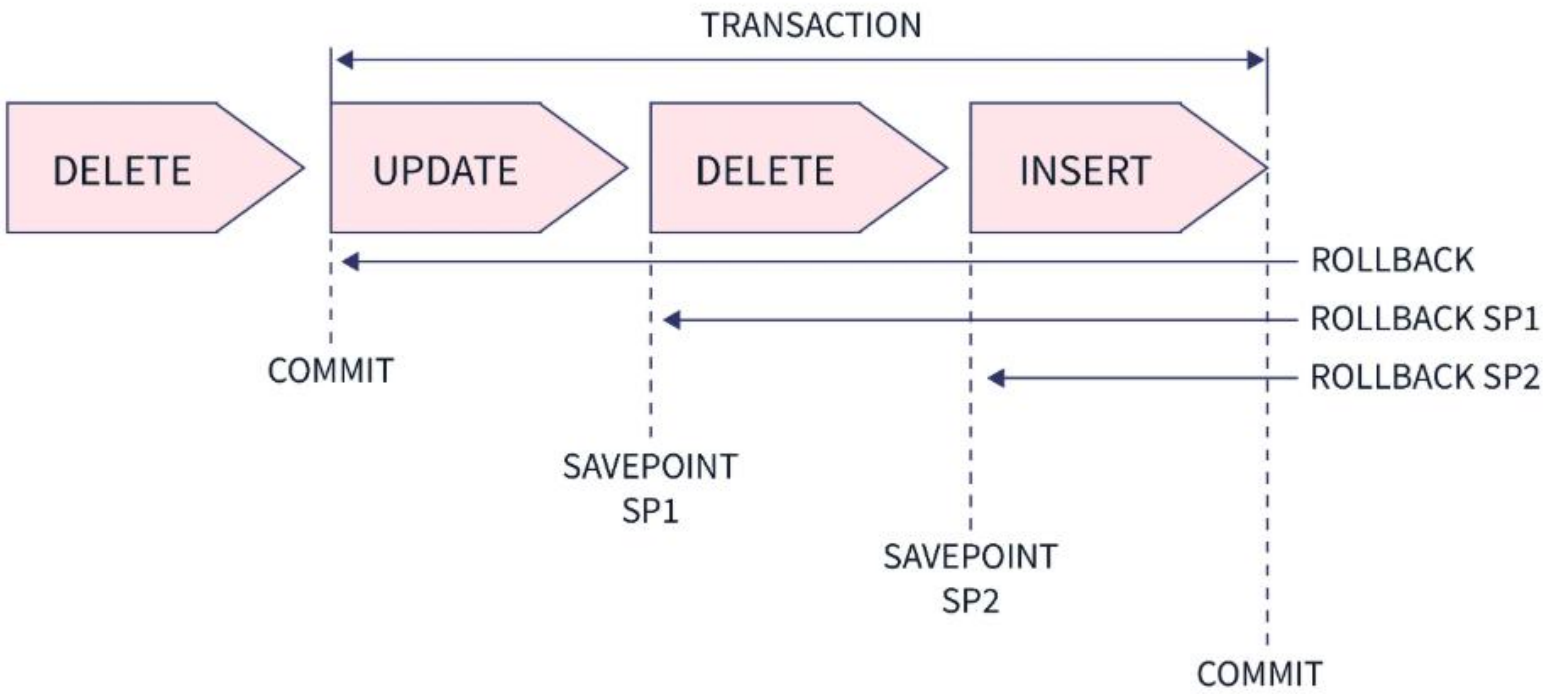
Rollback vs. Rollforward



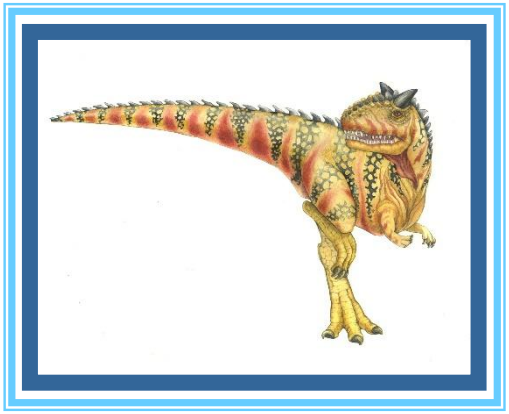
Rollback

Reverts data back to previous state

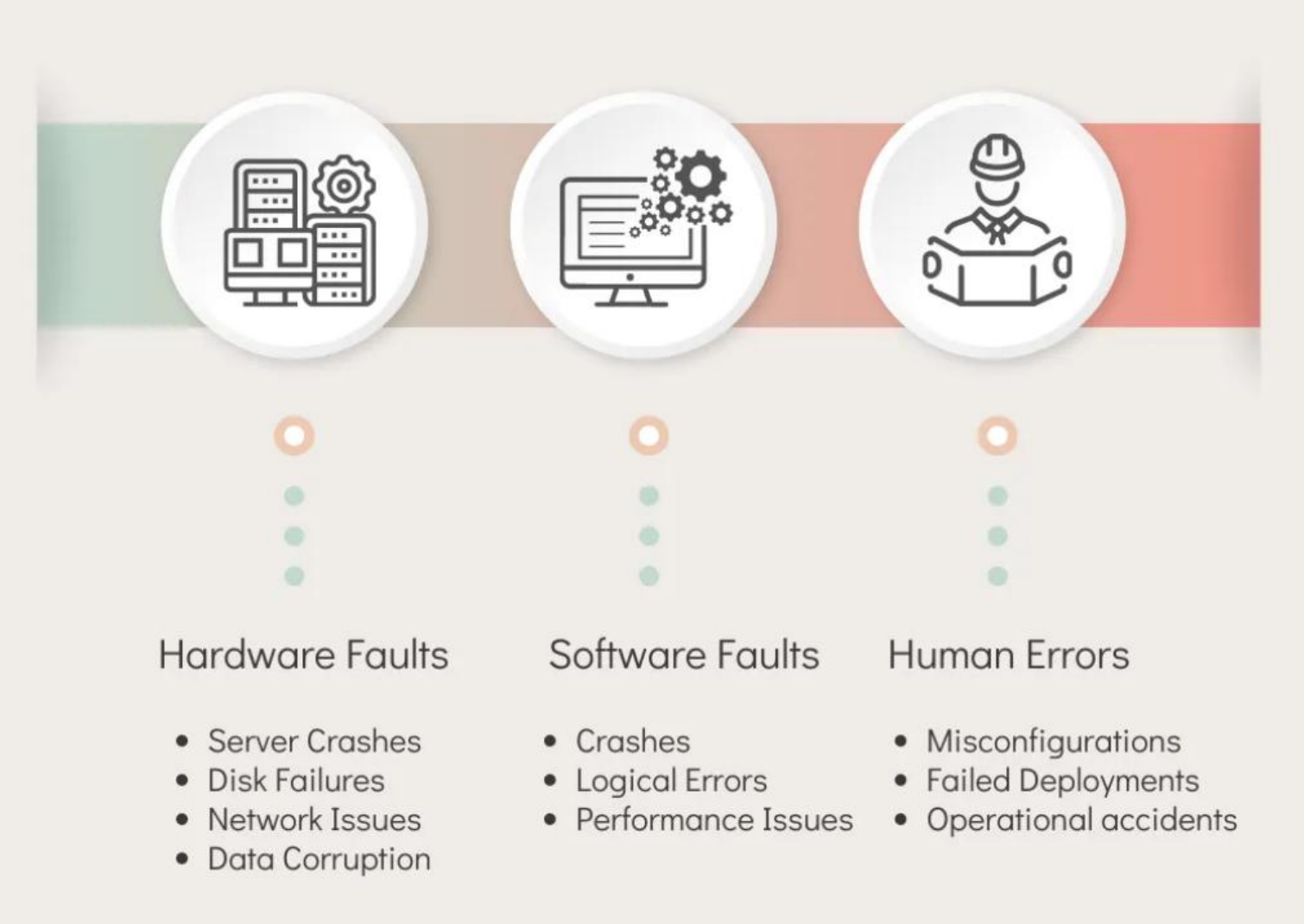
Rollback in Database Transactions



Data Reliability



Failures in Operating Systems



Data Reliability

Data Reliability:

The length of the statistically expected continuous span of time over which data stored by a population of identical storage subsystems can be correctly retrieved, expressed as Mean Time to Data Loss (MTDL).

Source: SNIA www.snia.org

Metrics

From Computer System:

MTTF (Mean Time to Failure)

MTTR (Mean Time to Repair)

MTBF (Mean Time Between Failures)

MTDL (Mean Time to Data Loss)

From I/O Devices:

FR (Failure Rate)

AFR (Annual Failure Rate)

ARR (Annual Repair Rate)

BER (Bit Error Rate)

UBER (Uncorrectable Bit Error Rate)

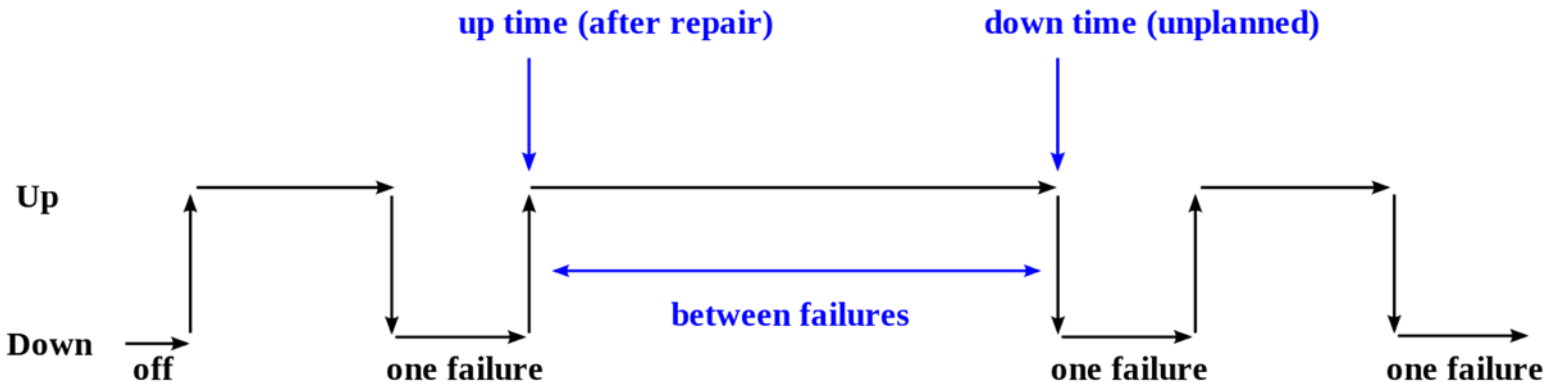
RBER (Raw Bit Error Rate)

Relationships among Various Metrics

MTBF=MTTF+MTTR **FR=1/MTTF**

Availability=MTTF/MTBF

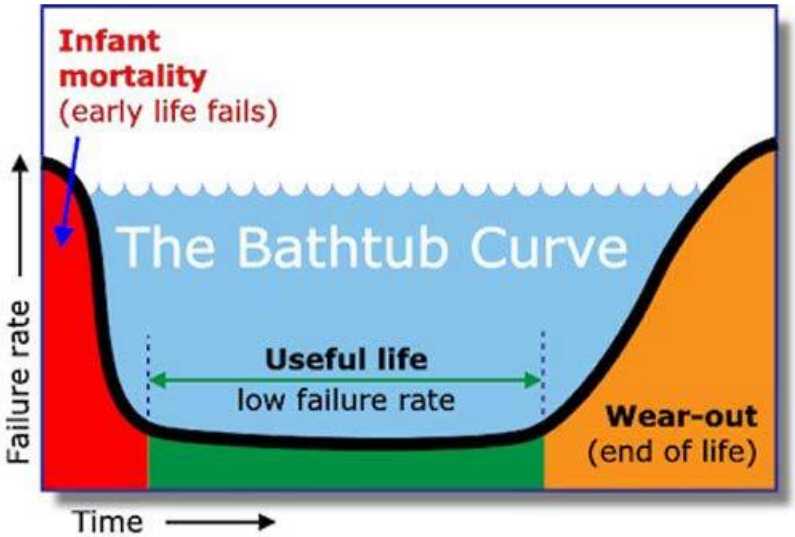
Reliability=MTTR/MTBF

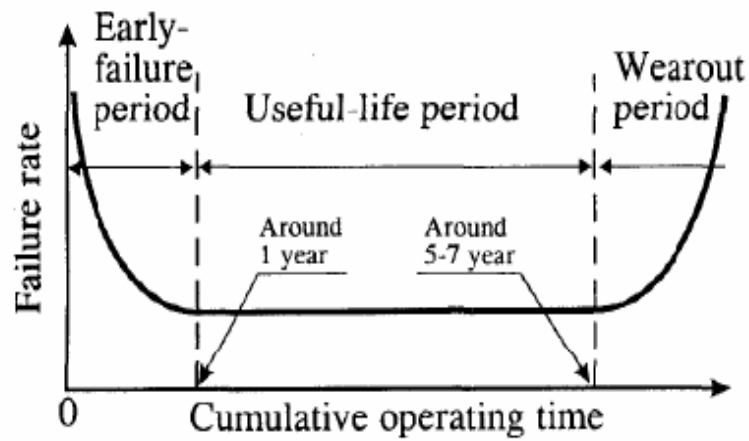


Time Between Failures = { down time - up time}

Lifecycle Failure Pattern for Computer Devices

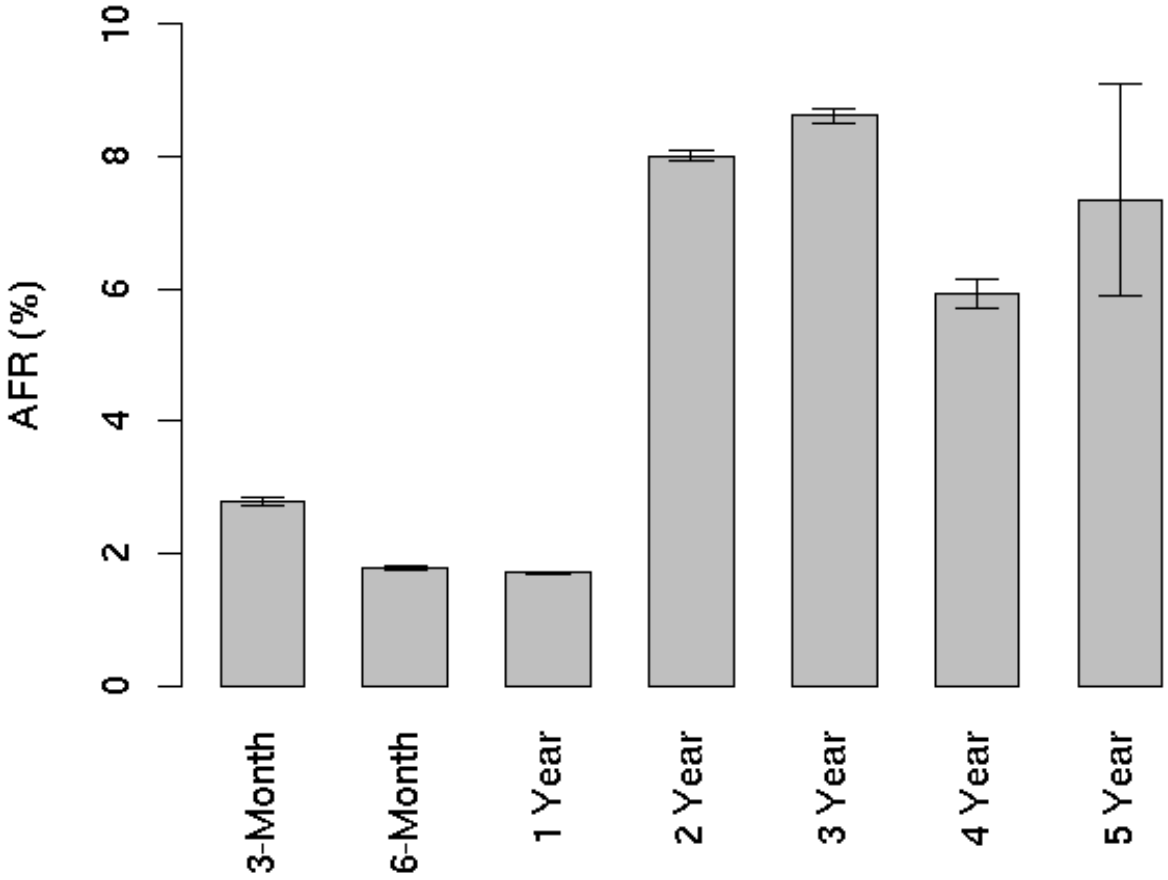
Typically follow a "**bathtub curve**" with high failure rates at the beginning (infant mortality) and the end (wear-out) of the lifecycle





HDD Failures in Large Scale Data Centers

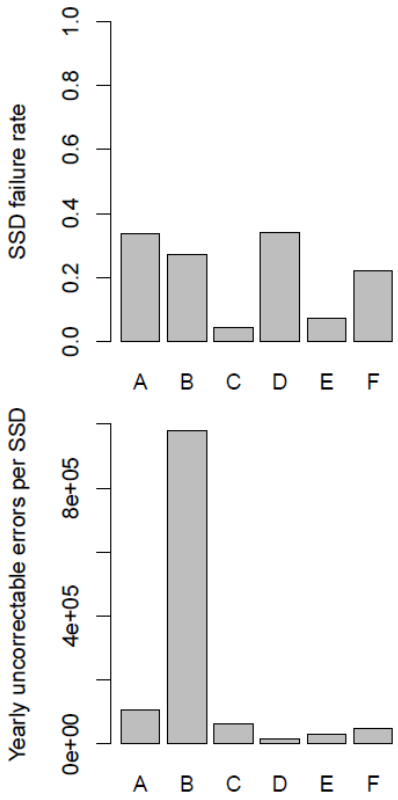
Google – Hard Disk Drive (HDD) Annual Failure Rate (1 million disks in five years)

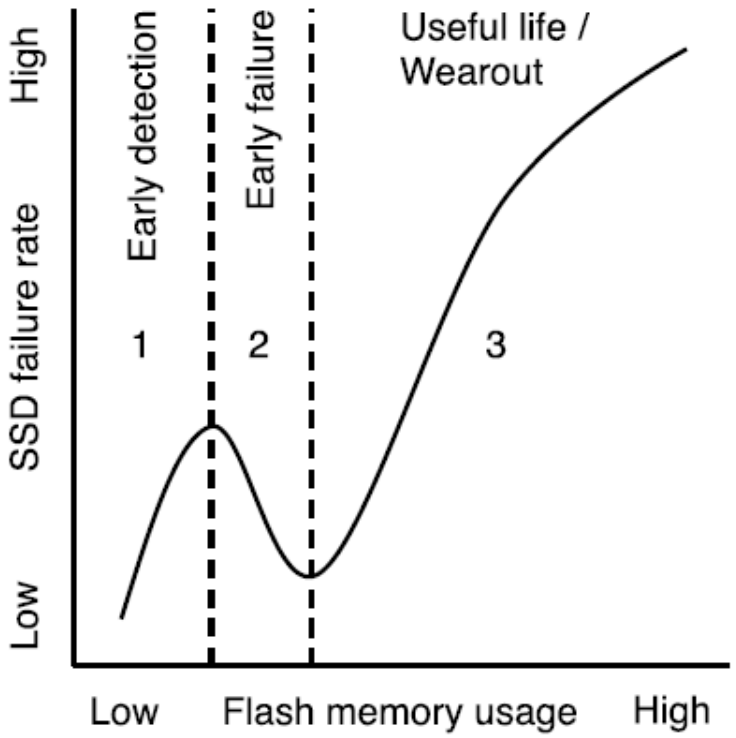


SSD Failures in Large Scale Data Centers (1)

Facebook (now Meta) - MLC SSDs

FR, UBER, lifetime (different from bathtub curve)

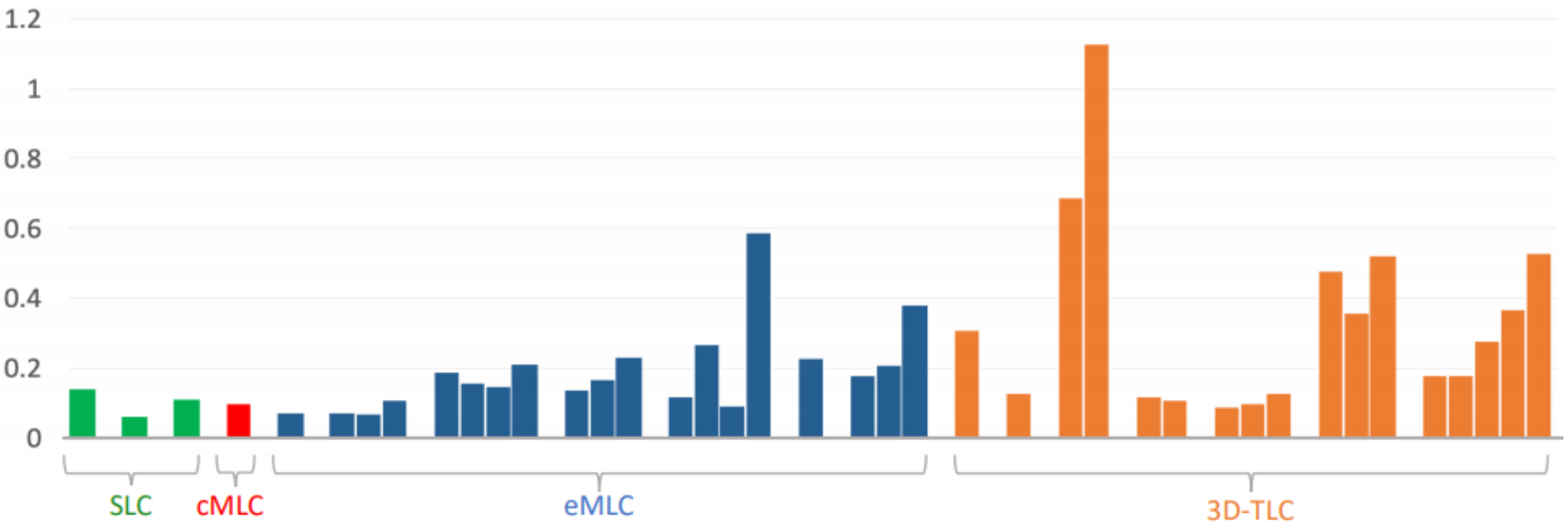




SSD Failures in Large Scale Data Centers (2)

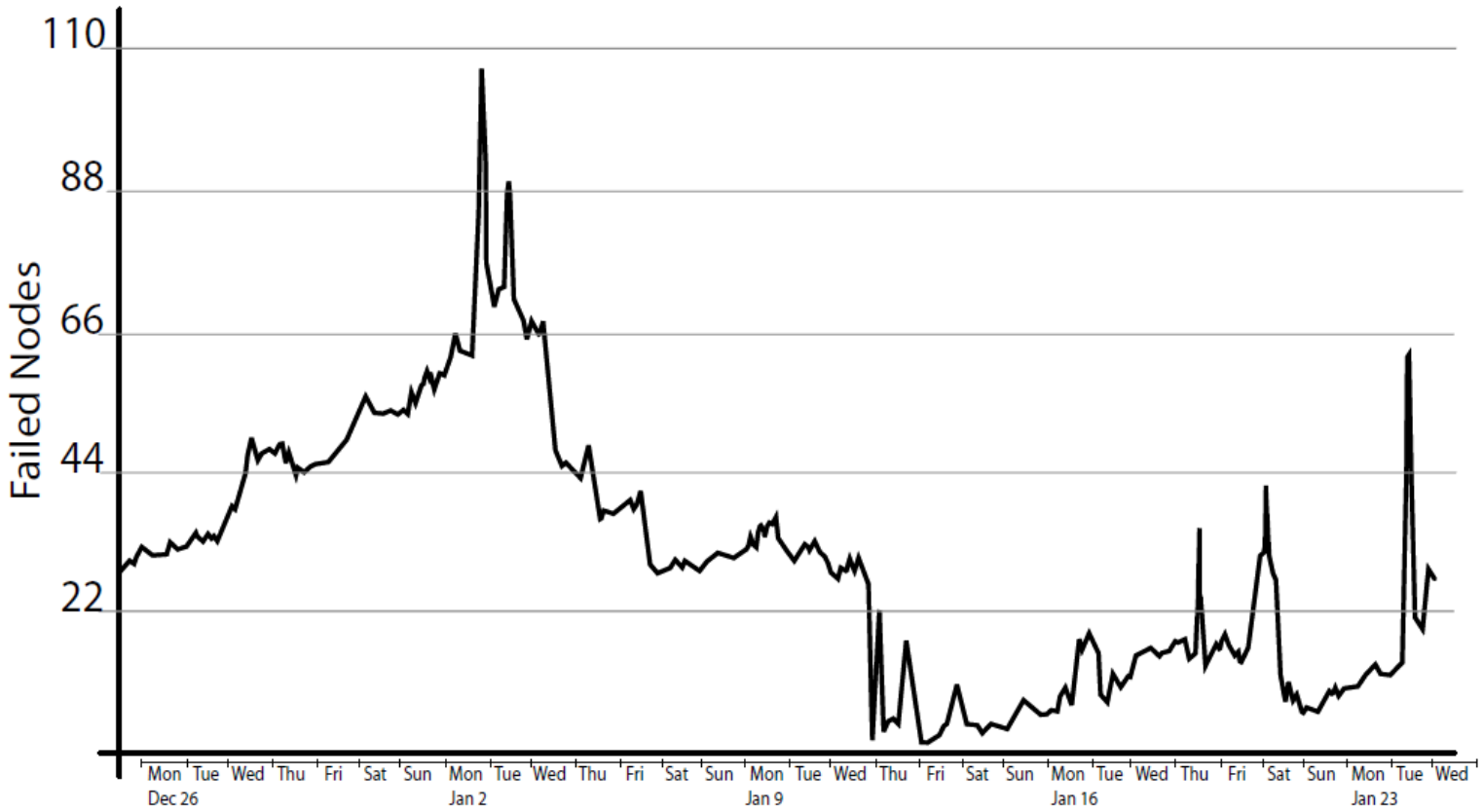
NetApp – 1.4 million SSDs (SLC, cMLC, eMLC, 3D-TLC)

Lower failure rates for SLC ($$$) versus cMLC/eMLC and 3D-TLC

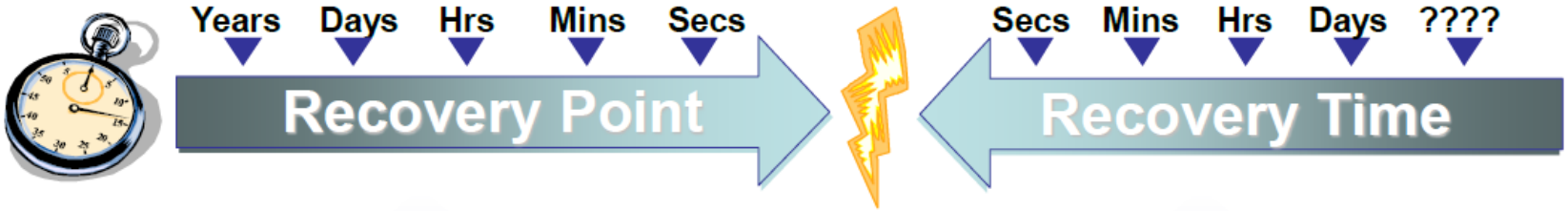


Node Failures in Large Scale Data Centers

Facebook – a 3000 node Hadoop cluster (one month usage)



Typical Data Protection Methods



Protection Methods

Tape Backups	Capture on Write	Synthetic Backup
Vaults	Disk Backups	Data Replication
Archival	Snapshots	Cloud Backup

Recovery Methods

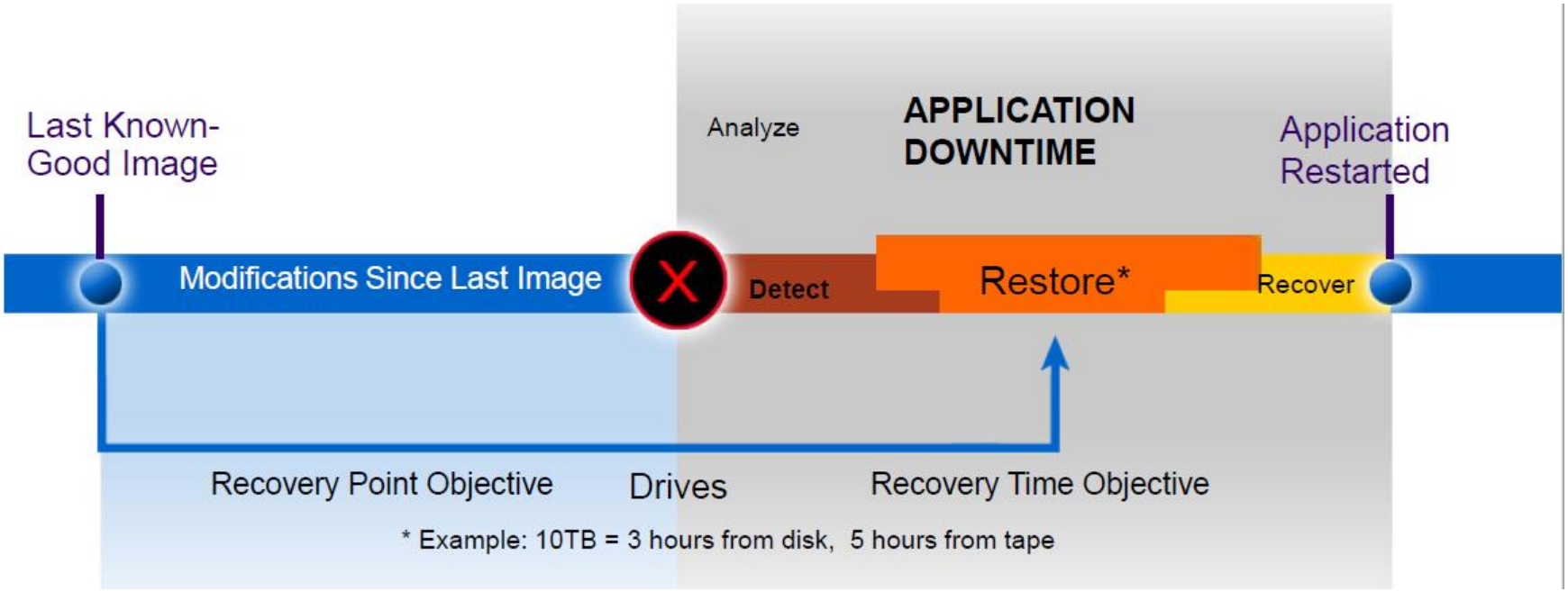
Instant Recovery Restore from Tape, Disk, Cloud

Point-in-Time Recovery

Roll Back

Search & Retrieve

Typical Data Recovery Process



Main Strategies

Redundancy with Original Data

- Snapshot
- Checkpointing

Redundancy without Original Data

- Replication
- Backup

Illustrated in Chapter 11

- Error Correction Code (e.g., ECC)
- Erasure Coding

Snapshot

A **snapshot** is a kind of “photo” of the state of a system (for example, a virtual machine or file system) at a specific point in time. If a failure occurs, you can quickly **roll back** and restore the system to this state.

Example of snapshots of a Btrfs filesystem

```
root@aventurine-topaz:~/ > snapper -c root list
 #  | Type   | Pre # | Date                     | User | Used Space | Cleanup | Description           | Userdata
----+--------+-------+--------------------------+------+------------+---------+-----------------------+----------------
 0  | single |       |                          | root |            |         | current               |
 1* | single |       | Tue Dec 31 12:47:24 2024 | root |   18.11 MiB |         | first root filesystem |
 2  | single |       | Tue Apr  8 14:27:47 2025 | root |   18.98 MiB |         | kernel 6.14.0         |
 3  | pre    |       | Tue Apr  8 16:05:40 2025 | root |    6.98 MiB | number  | zypp(zypper)          | important=yes
 4  | post   |     3 | Tue Apr  8 16:07:58 2025 | root |    1.38 MiB | number  |                       | important=yes
 5  | pre    |       | Tue Apr  8 16:16:33 2025 | root |  480.00 KiB | number  | zypp(zypper)          | important=no
 6  | post   |     5 | Tue Apr  8 16:16:38 2025 | root |  736.00 KiB | number  |                       | important=no
 7  | pre    |       | Wed Apr  9 09:26:14 2025 | root |    2.00 MiB | number  | zypp(zypper)          | important=no
 8  | post   |     7 | Wed Apr  9 09:26:36 2025 | root |    1.62 MiB | number  |                       | important=no
 9  | pre    |       | Wed Apr  9 14:28:34 2025 | root |    1.33 MiB | number  | zypp(zypper)          | important=no
10  | post   |     9 | Wed Apr  9 14:29:10 2025 | root |    1.56 MiB | number  |                       | important=no
11  | pre    |       | Thu Apr 10 08:47:09 2025 | root |    3.44 MiB | number  | zypp(zypper)          | important=no
12  | post   |    11 | Thu Apr 10 08:52:01 2025 | root |    4.09 MiB | number  |                       | important=no
```

How Snapshots Work?

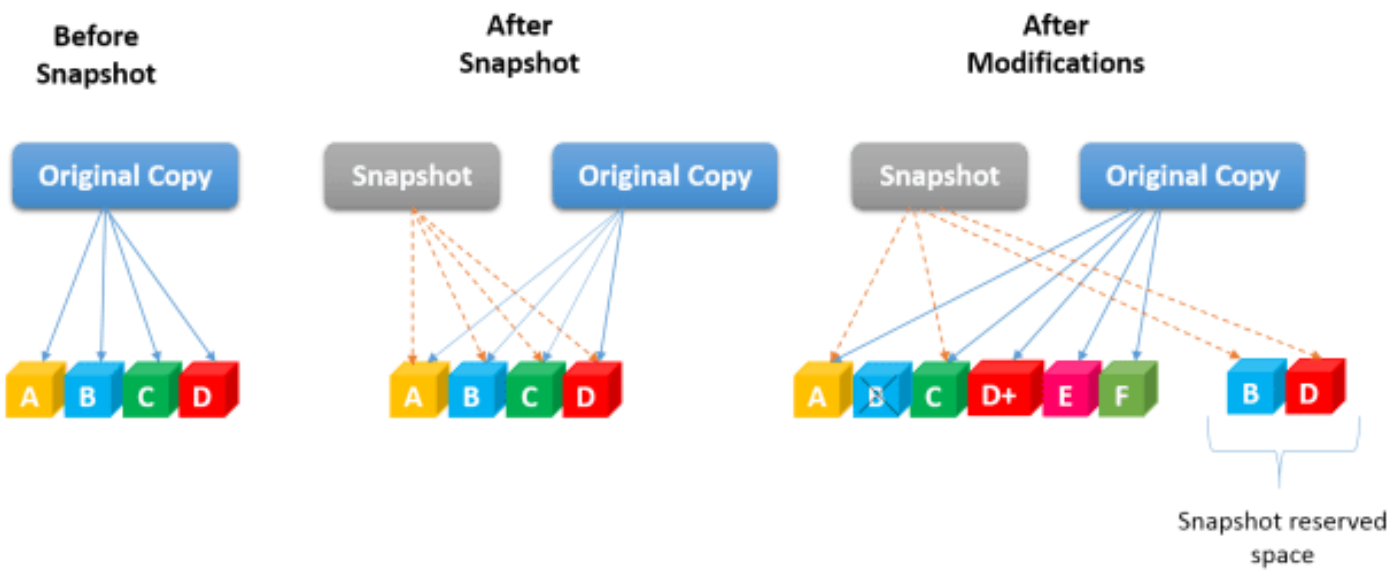
Features

- Online/Offline
- Combined with **roll back**
- Stores with original data
- Rapid recovery (~seconds)

Types of Snapshots

Copy-on-write snapshots

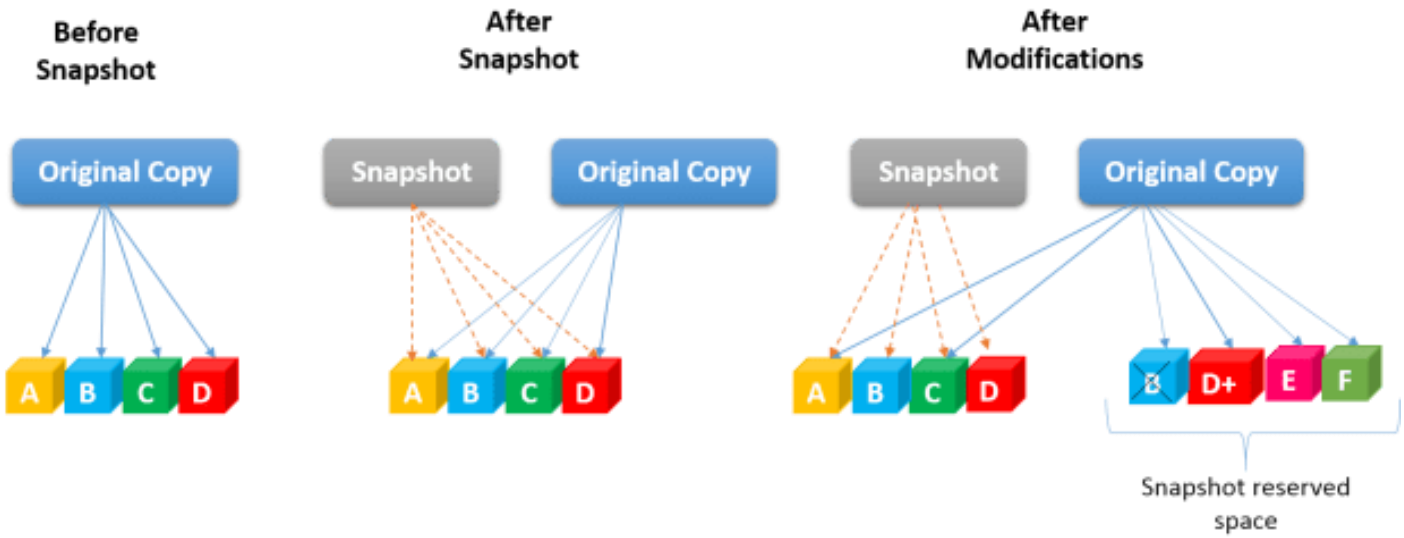
- Easy to implement
- Double write penalty



Types of Snapshots

Redirect-on-write snapshots

- Complex and fragmentations
- No write penalty



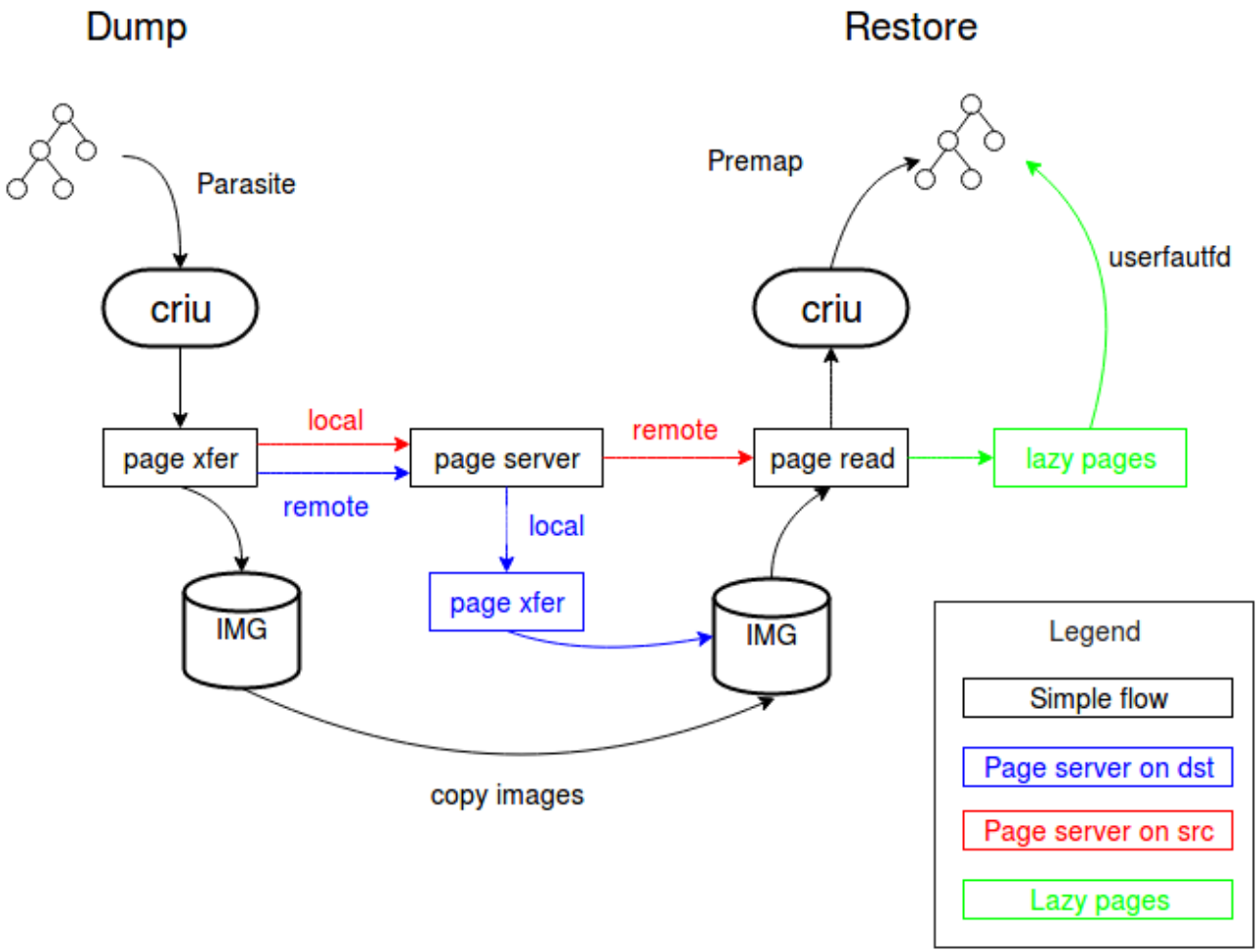
Checkpointing

Checkpointing is a technique that provides fault tolerance for computing systems. It involves saving a **snapshot** of an application's state, so that it can restart from that point in case of failure.

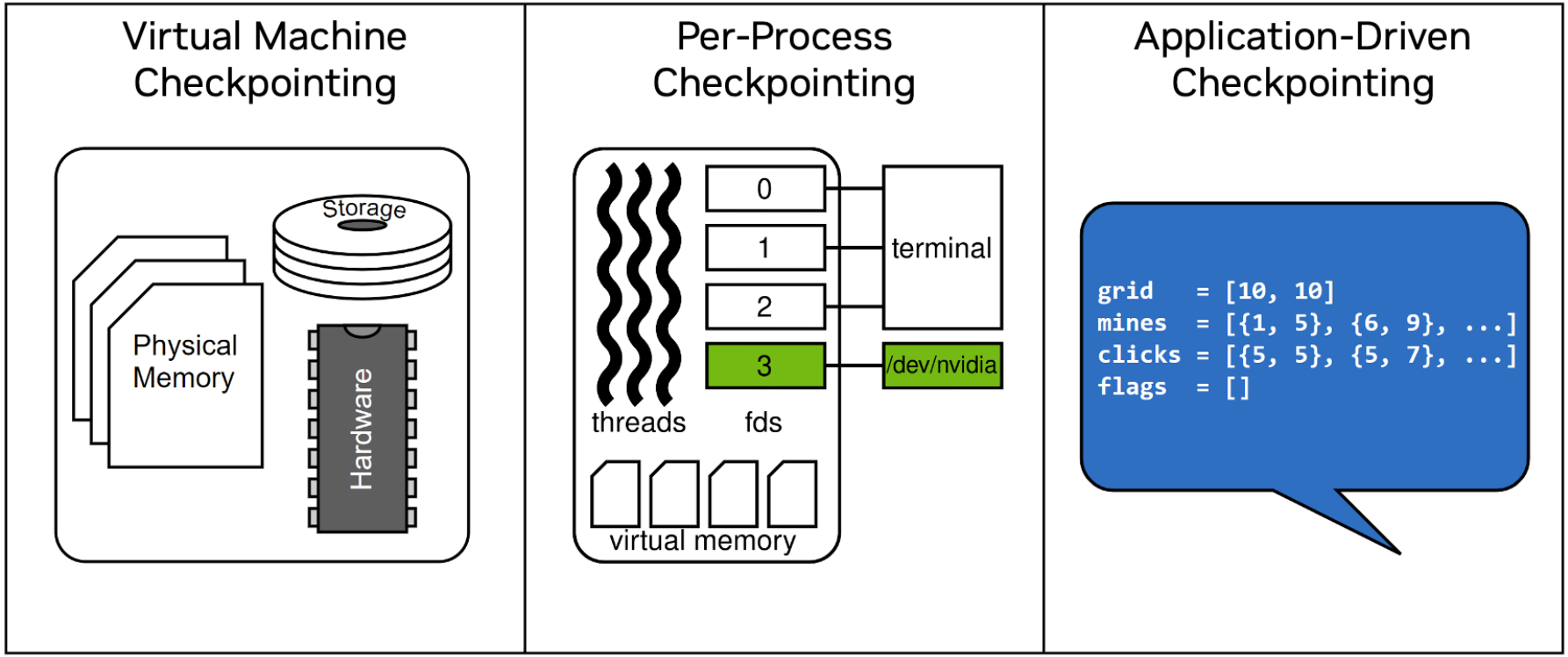
CRIU (Checkpoint/Restore in Userspace) in Linux

- Anonymous memory
- Threads
- Regular files
- Sockets
- Pipes between checkpointed processes

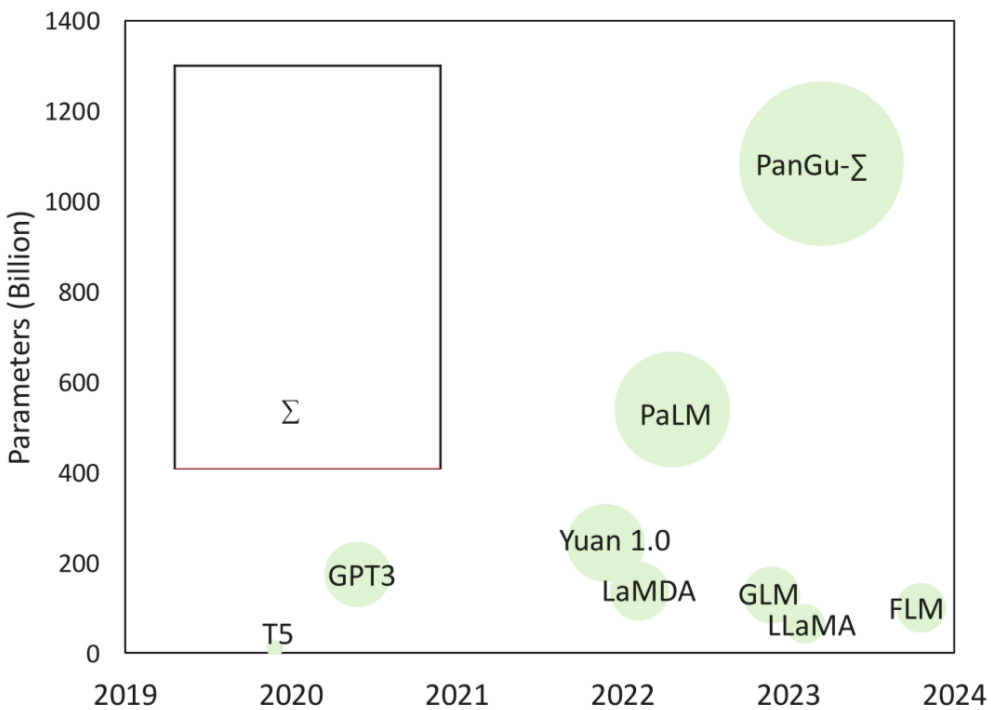
CRIU in Linux



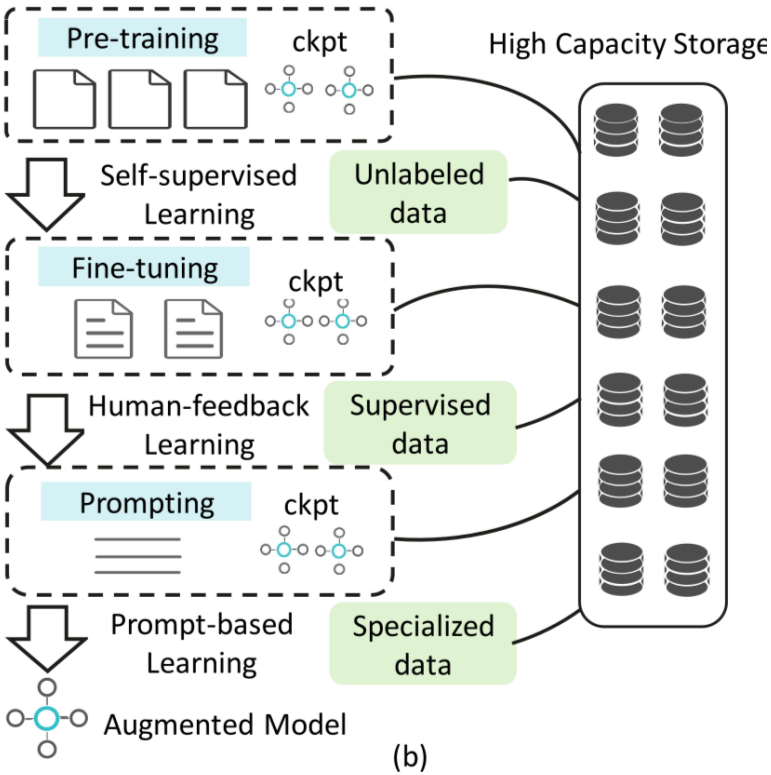
Types of Checkpointing



LLM Checkpointing (LLM ckpt)



(a)



(b)

Replication

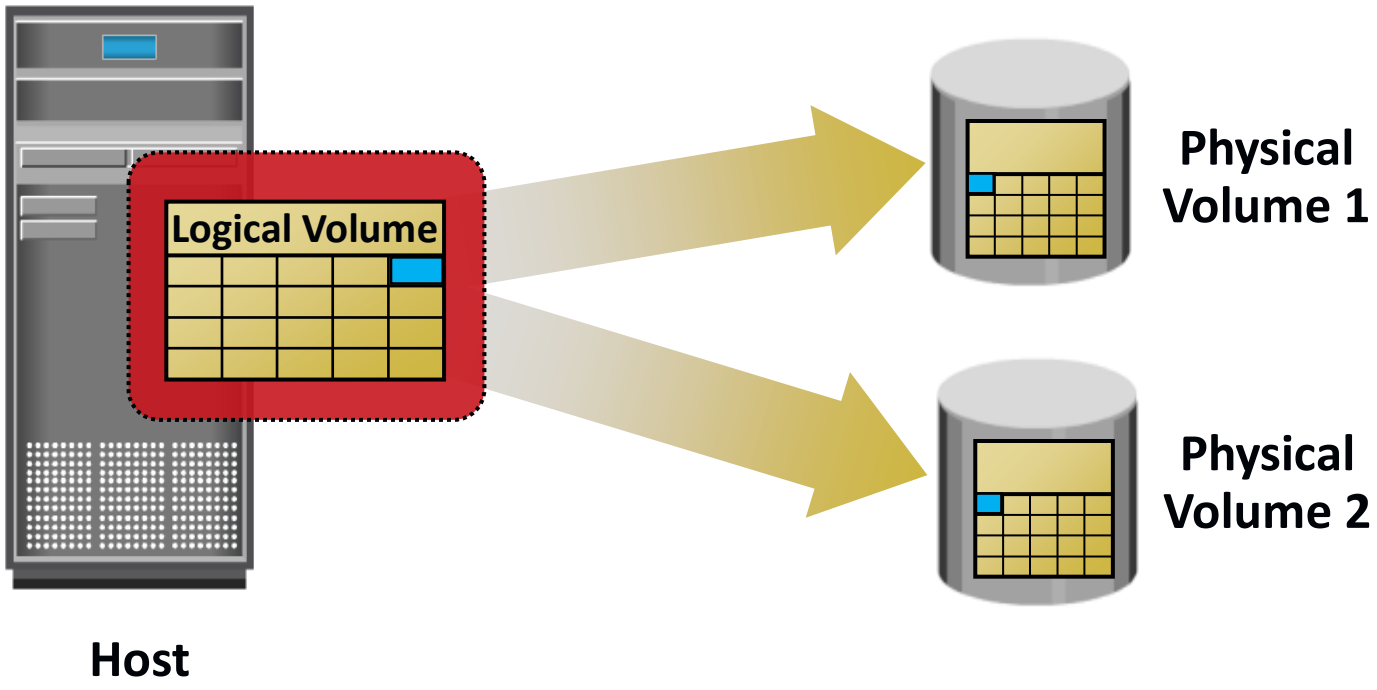
Replication:
It is a process of creating an exact copy (replica) of data.

Replication can be classified as

- Local replication
  - Replicating data within the same array or data center
- Remote replication
  - Replicating data at remote site

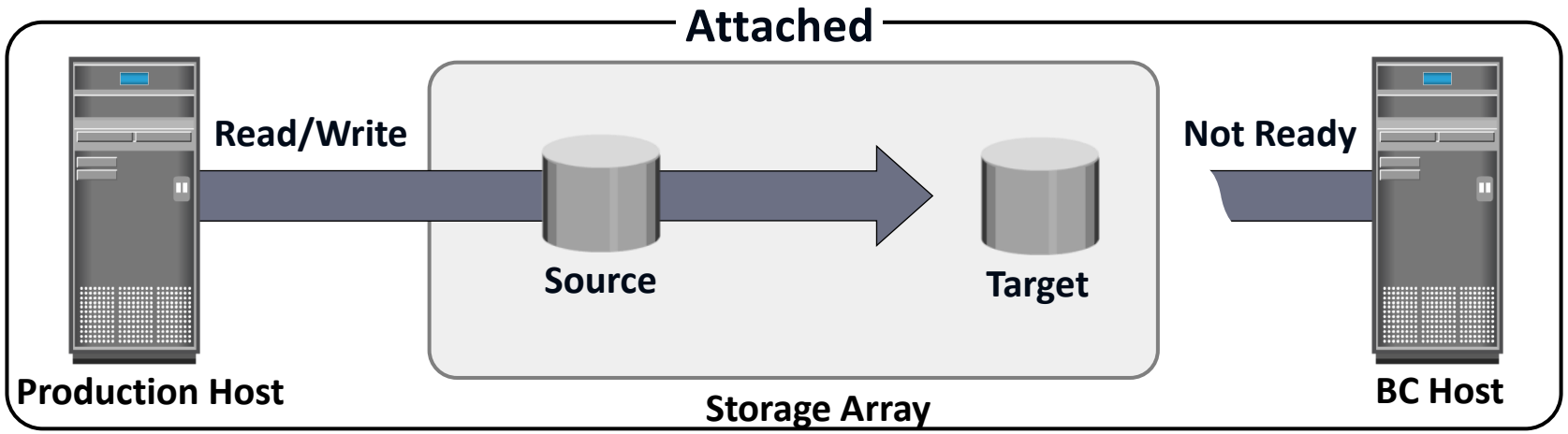
Local Replication (1)

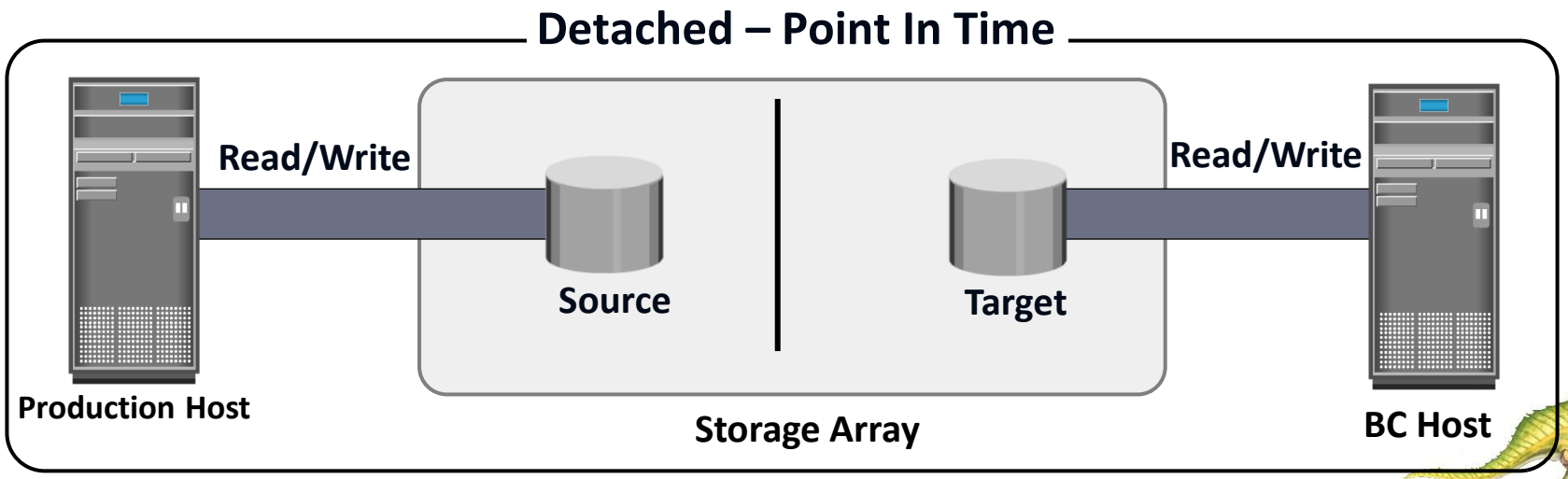
Logical Volume Manager (LVM) -based Mirroring



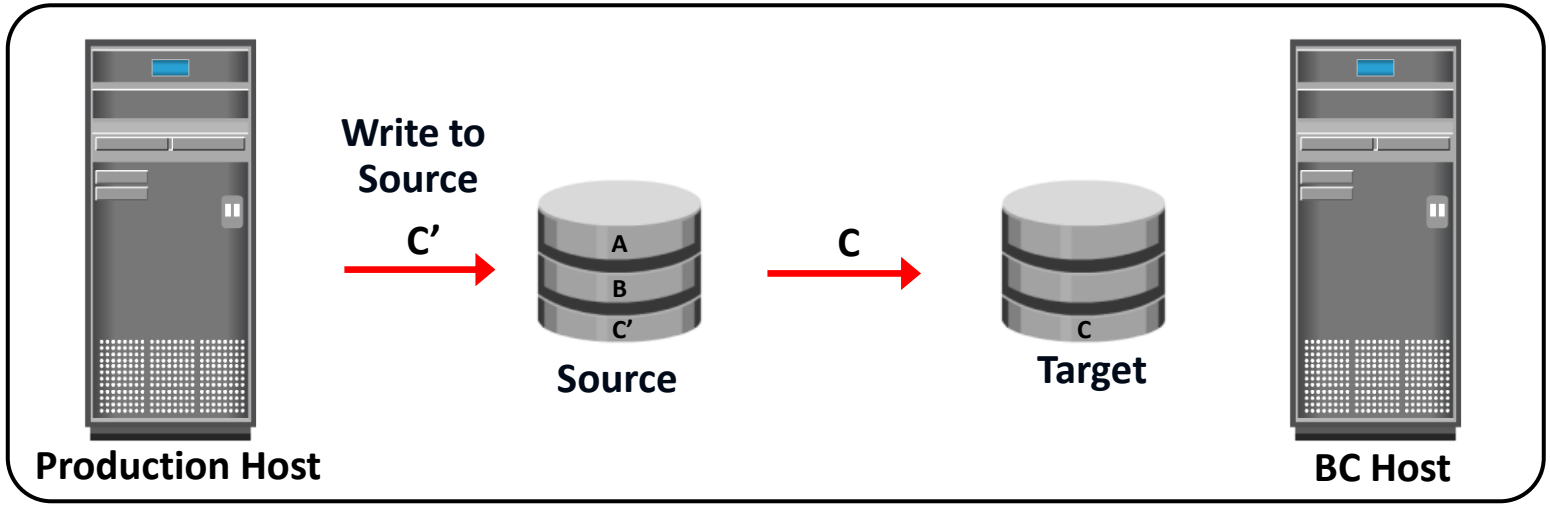
Local Replication (2)

Full-Volume Mirroring





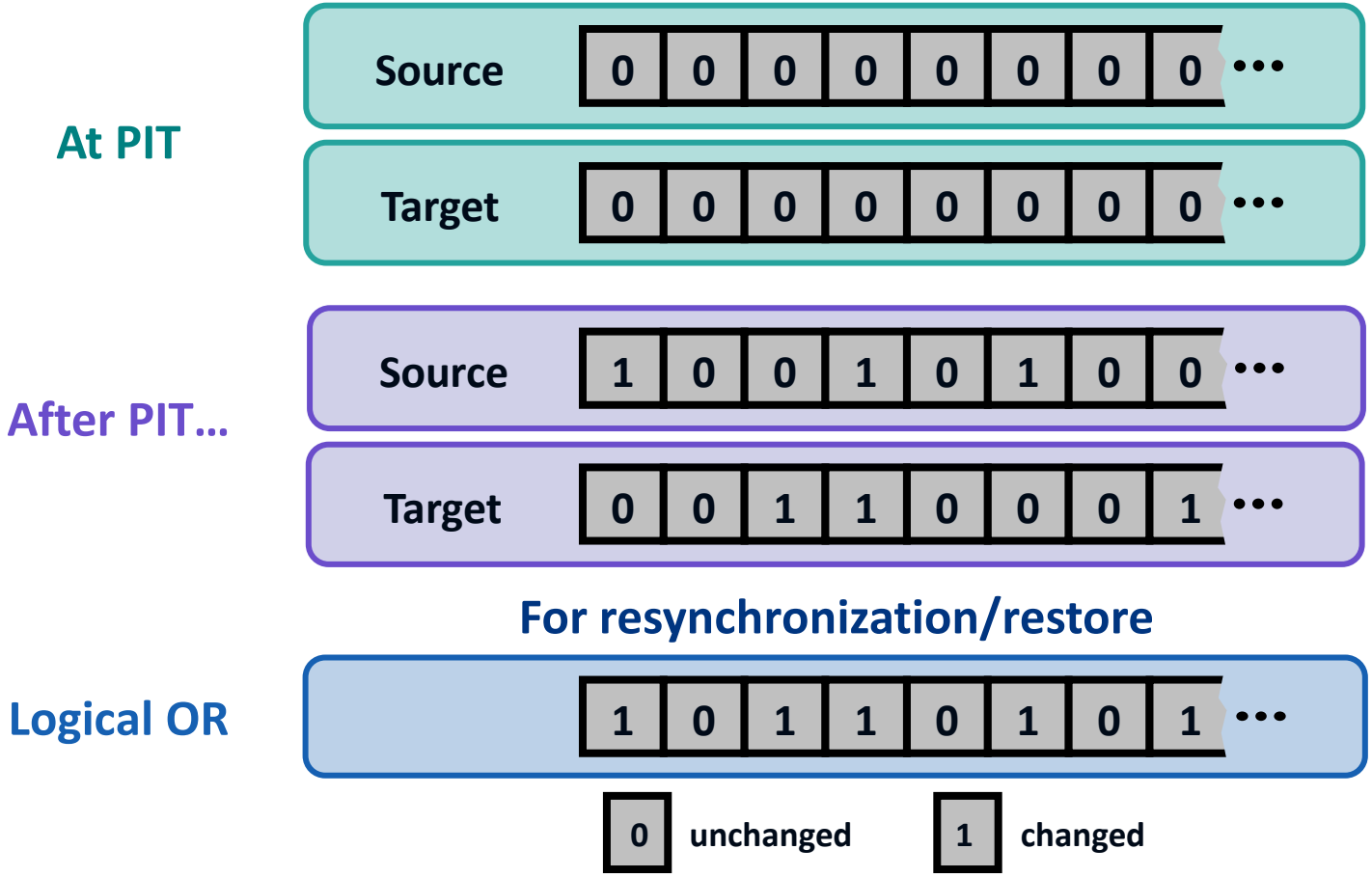
Copy on First Access: Write to the Source



When a write is issued to the source for the first time after replication session activation:

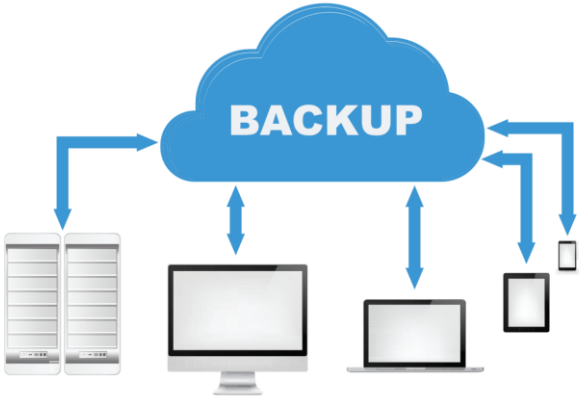
- Original data at that address is copied to the target
- Then the new data is updated on the source
- This ensures that original data at the point-in-time of activation is preserved on the target

Tracking Changes to Source and Target

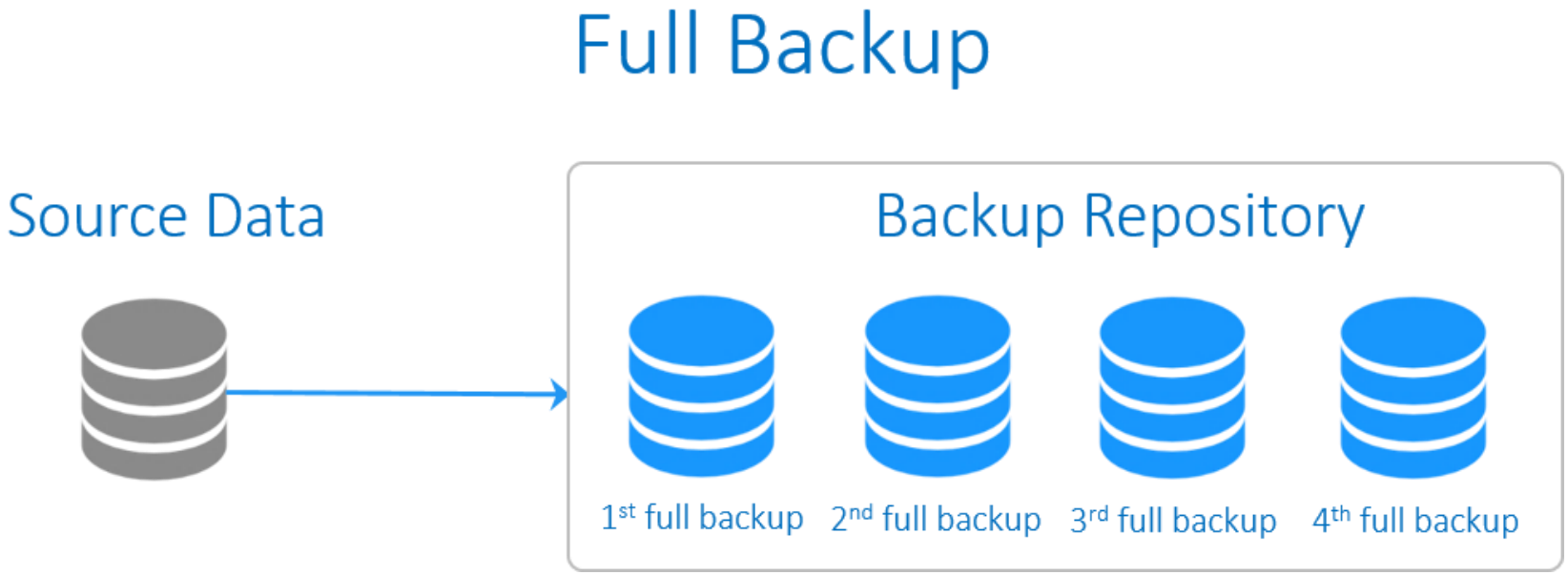


Traditional Backup Strategies

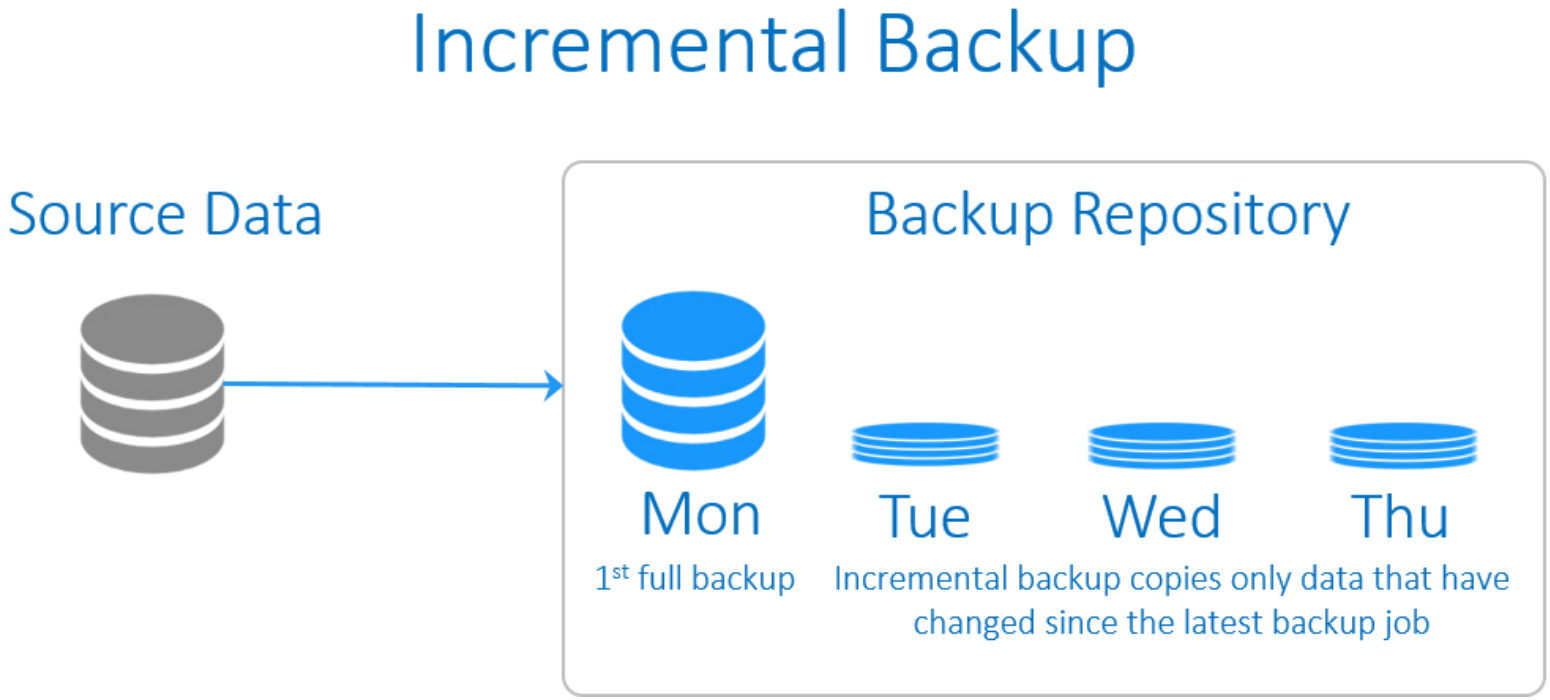
- Full Backup
  - Everything copied to backup (cold or hot backup)
  - Huge resource consumption (server, network, tapes)
- Incremental Backup
  - Only the data that changed since last full or incremental
  - Much less data is transferred
- Differential Backup
  - All of the data that changed from the last full backup
  - Usually less data is transferred than a full
  - Usually less time to restore full dataset than incremental



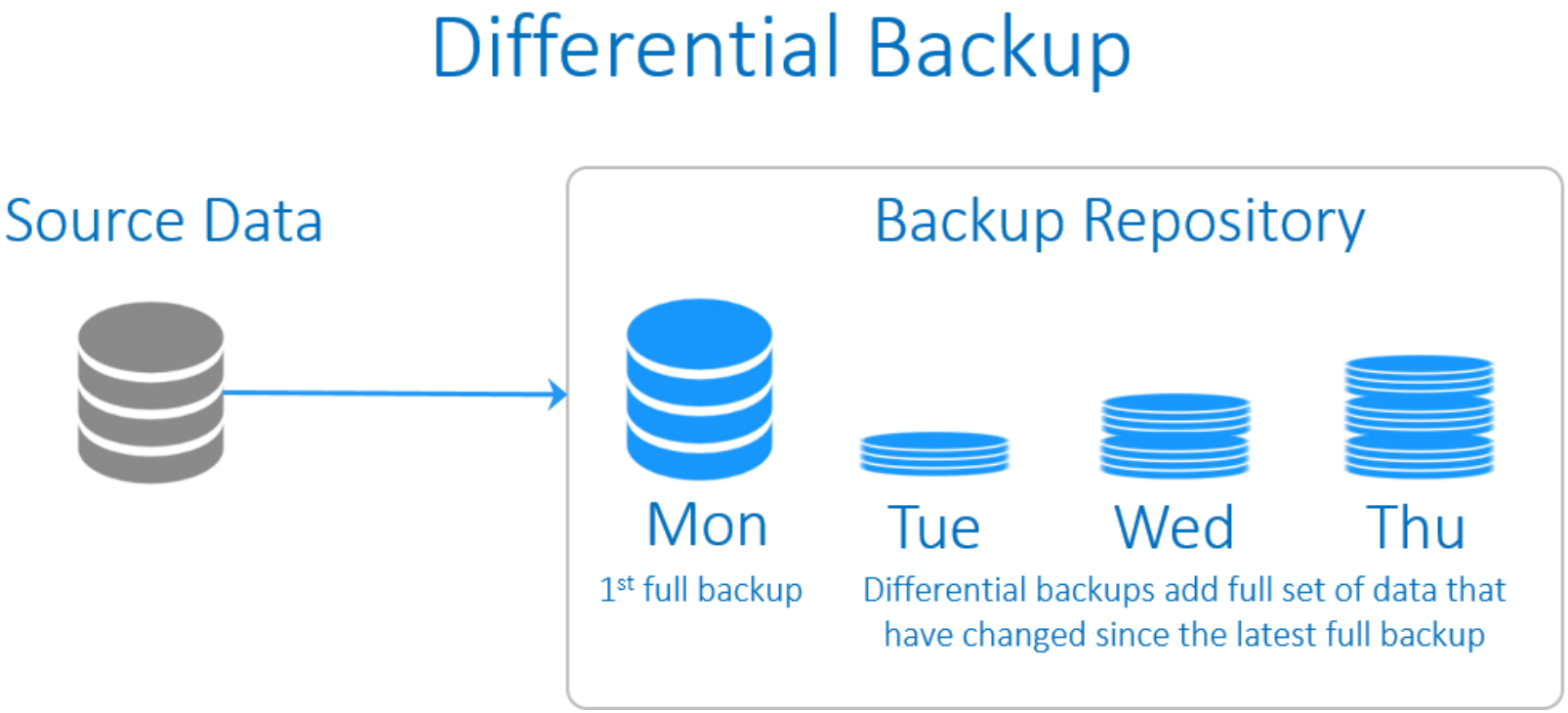
Full Backup


Full Backup
Source Data
Backup Repository
1st full backup
2nd full backup
3rd full backup
4th full backup

Incremental Backup



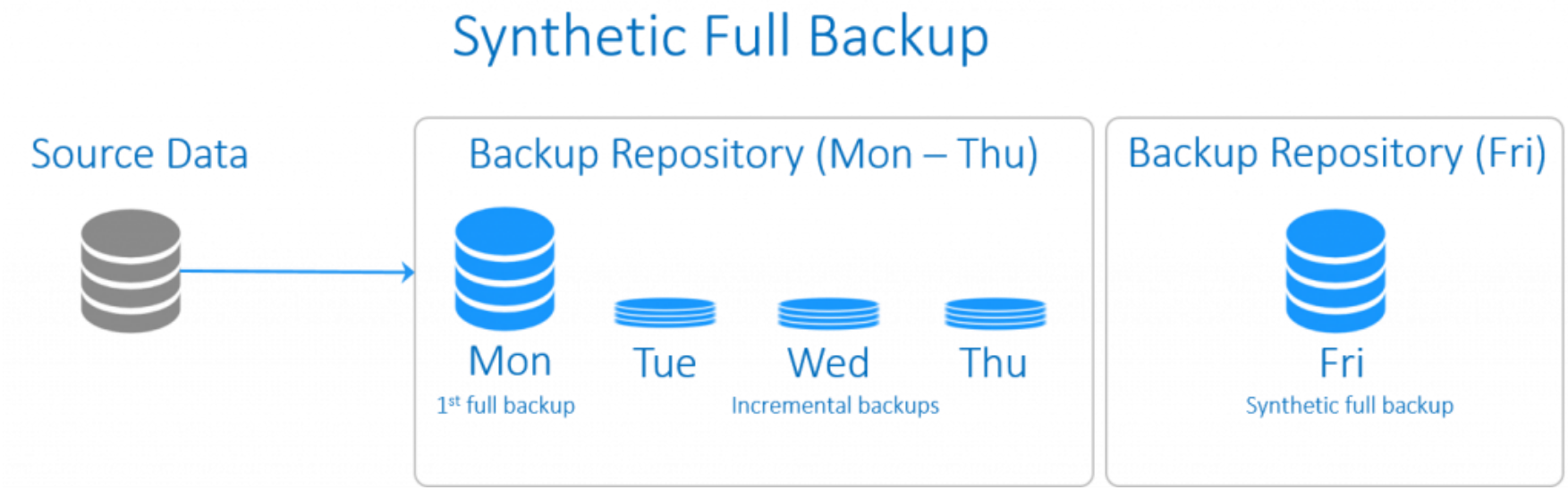
Differential Backup



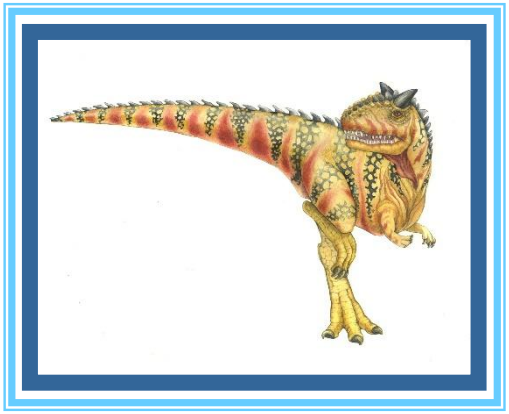
Advanced Backup Strategy – Synthetic Full Backup

Synthetic Full Backup

Incremental Backups are performed each day



Homework

- Exercises at the end of Chapter 8 (OS book)
  - 8.3, 8.9, 8.18, 8.27, 8.28

End of Chapter 8